



6CCS3PRJ

Discrete Math Tutor: Binary Relations

Final Project Report

Author: 

Supervisor: 

Student ID: 

April 5, 2023

Abstract

Interactive-based learning offers an exciting opportunity to improve student engagement, motivation and retention. With the advancement of technology, it became feasible to integrate interactivity in online learning platforms. Such platforms have the potential in significantly enhancing the quality of education and improving the students' learning experience. Therefore, the goal of this project is to provide an interactive and effective approach to teaching binary relations, a foundational topic in Discrete Mathematics. The project builds a web-based system that features three interactive learning tools. These tools include an interactive visualiser for visualising binary relations, an interactive simulator for simulating the Warshall algorithm, and an interactive explorer tool for exploring the various types and properties of binary relations. By integrating several interactive elements within these tools, the project seeks to engage students with the subject and enhance their overall learning outcomes.

Originality Avowal

I verify that I am the sole author of this report, except where explicitly stated to the contrary.

I grant the right to King's College London to make paper and electronic copies of the submitted work for purposes of marking, plagiarism detection and archival, and to upload a copy of the work to Turnitin or another trusted plagiarism detection service. I confirm this report does not exceed 25,000 words.



April 5, 2023

Acknowledgements

I would like to express my appreciation and gratitude to my supervisor, 
, for their valuable supervision and assistance. Their insightful feedback coupled with their experience in the field has been a key factor in shaping the success of this project. Also, their encouraging words were a great source of motivation, inspiring me to push forward and constantly refine my work to a high standard. I am honored and truly fortunate to have worked under their mentorship.

Contents

1	Introduction	3
1.1	Motivations	4
1.2	Scope	4
1.3	Objectives	5
2	Background and Context	6
2.1	Brief	6
2.2	Representations of Binary Relations	6
2.3	Properties of Binary Relations	7
2.4	Special Types of Binary Relations	8
2.5	Hasse Graph	9
2.6	The Warshall Algorithm	11
3	Literature Review	13
3.1	Brief	13
3.2	Web-based Interactive learning	13
3.3	Review of Existing Work	15
3.4	Extending Existing Tools: Enhancing Capabilities and Overcoming Limitations	17
4	Requirements & Specification	19
4.1	Brief	19
4.2	Functional Requirements	19
4.3	Non-Functional Requirements	24
4.4	Specification	25
5	Design	28
5.1	Brief	28
5.2	System Architecture	28
5.3	Use Cases	29
5.4	Function-Based Design: A Modular Approach	30
5.5	Software Design Pattern	31
5.6	Data Flow	31
5.7	Data Management	32
5.8	Graphical User Interface	33

6	Implementation	42
6.1	Brief	42
6.2	Development Approach	42
6.3	Code Structure	44
6.4	Saving, Deleting and Retrieving Relations	45
6.5	Parsing Relations	46
6.6	Relation Visualiser Tool	47
6.7	Warshall Simulator Tool	50
6.8	Relation Explorer Tool	56
7	Testing	70
7.1	Brief	70
7.2	Automated Unit Testing	70
7.3	Manual Testing	73
7.4	Integrated Test Suite	74
8	Evaluation	75
8.1	Brief	75
8.2	Performance Evaluation	75
8.3	Accessibility Evaluation	77
8.4	Compatibility Evaluation	78
8.5	Critical Appraisal: Evaluating Strengths and Areas of Improvement	80
9	Professional and Ethical Issues	82
10	Conclusion and Future Work	83
10.1	Conclusion	83
10.2	Future Work	84
	Bibliography	87

Chapter 1

Introduction

The effectiveness of passive and active learning styles remains a widely debated topic within the academic community. Despite the continued prevalence of passive learning methods, there is a recent shift towards interactive teaching techniques that prioritise student-centered learning. Traditionally, interactivity in any educational setting is depicted solely as teacher-student or student-student interactions. However, the advancement of technology has revolutionized the way we approach education and introduced novel forms of interactivity, including student-computer interactions. According to the Oxford Online Dictionary, interactivity is defined as “the fact of allowing information to be passed continuously and in both directions between a computer or other device and the person who uses it” [1]. With the help of the online technology, students learning quality can be enhanced through the design of interactive-based learning systems that foster high levels of engagement, retention, and critical thinking. Subjects like Discrete Mathematics can aid from such systems as it is heavily reliant on theory-based concepts which are better understood when incorporated with interactive elements. The inclusion of these interactive elements in the teaching of Discrete Mathematics can provide students with a hands on learning experience; thereby bridging the gap between theory and practice.

1.1 Motivations

Learning and teaching during the pandemic caused by COVID-19 was a huge transformational phase in education. The change from face-to-face to online classrooms, which lack interactions with tutors and peers, has been a source of stress and demotivation for many students, myself included. To overcome these challenges and remain engaged with distant learning, I relied on various tools and interactive-learning platforms to reinforce the concepts learnt in the online-classes and pre-recorded lectures. I realised how these interactive tools were very advantageous as it provided a playful and experimental learning environment, which enhanced my understanding of the material.

My past experience with distant learning fueled my motivation to undertake this project, allowing me to transform the value of interactive tools to create an interactive learning platform tailored to the objectives of this project. Furthermore, my passion for web development has been a key driver to make this platform web-based. Choosing a web-based system not only offers a convenient learning experience to students but also allowed me to build my knowledge and skills in the field of web development. Moreover, developing this platform requires learning JavaScript, which is the industry leader in designing dynamic and interactive websites. Taking the opportunity and challenge to learn a new language is another reason to why I was motivated to undertake this project.

1.2 Scope

This project is designed to provide students with an interactive learning experience on one of the most fundamental topics in Discrete Mathematics: Relations. Generally, relations can be n -ary, however, this project will focus particularly on binary relations.

The scope can be defined by this list of subtopics:

1. Representations of binary relations.
2. Properties of binary relations.
3. Partial order relations and Hasse graphs.
4. Linear order relations.
5. Equivalence relations.
6. The Warshall algorithm.

1.3 Objectives

The main objective of this project is to build an interactive web-based system that has three learning tools for teaching binary relations:

1. Interactive tool for visualising the different binary relations representations.
2. Interactive tool for simulating the Warshall algorithm.
3. Interactive tool for exploring the properties and types of binary relations.

The aim of these tools is to effectively engage students and provide them with a baseline knowledge and practice zone on the topics mentioned earlier. The tools can be accessed by visiting our website through the link: 

Chapter 2

Background and Context

2.1 Brief

The primary objective of this chapter is to provide a detailed background overview of the key terminologies as well as the fundamental concepts and principles related to binary relations.

2.2 Representations of Binary Relations

A set in mathematics is a collection of distinct elements forming a group. For example, if we have the following set $A = \{1,2,3,4\}$ then we say that A is the set and 1, 2, 3, 4 are the elements of the set. The Cartesian Product of any two sets X and Y denoted as $X \times Y$ is the set of all ordered pairs (x, y) where x is an element of X and y is an element of Y which can be denoted as: $X \times Y = \{(x, y) \mid x \in X \text{ and } y \in Y\}$ [2]. For example, if $X = \{A, B, C\}$ and $Y = \{1, 2, 3\}$ then the Cartesian Product $X \times Y = \{(A, 1), (A, 2), (A, 3), (B, 1), (B, 2), (B, 3), (C, 1), (C, 2), (C, 3)\}$. A binary relation R on a set A is a collection of ordered pairs (x, y) of the Cartesian Product of the set $A \times A$. Therefore, we can say that R is a binary relation on A if and only if $R \subseteq A \times A$. We use the notation xRy to denote that $(x, y) \in R$ and we say that x is R-related to y [3]. Binary relations can be represented in three different ways: ordered pairs, directed graph or matrix. For example, if $A = \{1,2,3\}$ then we can represent a binary relation R in ordered pairs as follows: $R = \{(1,1), (1,2), (2,1), (2,2), (3,2), (3,3)\}$

The relation R can be represented by a directed graph as follows:

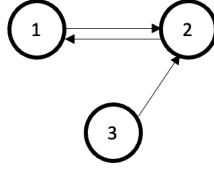


Figure 2.1: Directed Graph Representation of Relation R

It is worth noting that there are different ways to represent reflexive pairs (e.g., $(1,1)$) in a directed graph which could be either nodes with self-looping edges (i.e. an edge that connect a node to itself) or nodes with bolded borders (as shown in the figure above). This project, however, will consistently use the second representation to indicate reflexive pairs.

The relation R can be represented by a matrix as follows:

$$\begin{array}{c}
 \mathbf{1} \quad \mathbf{2} \quad \mathbf{3} \\
 \begin{array}{c} \mathbf{1} \\ \mathbf{2} \\ \mathbf{3} \end{array} \left[\begin{array}{ccc} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{array} \right]
 \end{array}$$

Figure 2.2: Matrix Representation of Relation R

2.3 Properties of Binary Relations

Binary relations can have any of the following properties: reflexivity, symmetry, antisymmetry, and transitivity.

- A relation R on set X is reflexive if $(x, x) \in R$ for all $x \in X$ [4].

For example, if $X = \{1,2,3\}$ then a reflexive relation could be $R = \{(1,1), (2,2), (3,3)\}$.

- A relation R on set X is symmetric if $(y, x) \in R$ whenever $(x, y) \in R$ [4].

For example, if $X = \{1,2,3\}$ then a symmetric relation could be $R = \{(1,1), (2,1), (1,2), (3,3)\}$.

- A relation R on set X is antisymmetric if $(y, x) \notin R$ whenever $(x, y) \in R$ and $x \neq y$ [4].

For example, if $X = \{1,2,3\}$ then an antisymmetric relation could be $R = \{(1,1), (2,1), (2,2), (3,3)\}$.

- A relation R on set X is transitive if $(x, z) \in R$ whenever $(x, y) \in R$ and $(y, z) \in R$ [4].

For example, if $X = \{1, 2, 3\}$ then a transitive relation could be $R = \{(1, 1), (1, 2), (2, 3), (1, 3)\}$

2.4 Special Types of Binary Relations

2.4.1 Equivalence Relations

An equivalence relation is a relation that is reflexive, symmetric, and transitive [4]. The ‘is equal to’ denoted as ‘=’ on the set of real numbers is an example of an equivalence relation since for all $x, y, z \in \mathbb{R}$ these three conditions hold:

1. Reflexivity ($x = x$)
2. Symmetry (if $x = y$ then $y = x$)
3. Transitivity (if $x = y$ and $y = z$ then $x = z$)

2.4.2 Partial Order Relations

A partial order relation is a relation that is reflexive, antisymmetric, and transitive [4]. The ‘divisibility’ relation on the set of natural numbers is an example of a partial order relation since for all $x, y, z \in \mathbb{N}^+$ these three conditions hold:

1. Reflexivity (x is divisible by itself).
2. Antisymmetry (if x is divisible by y and y is divisible by x then $x = y$).
3. Transitivity (if x is divisible by y and y is divisible by z then x is divisible by z).

2.4.3 Linear Order Relations

A linear order relation R on set X is a partial order relation and satisfies the linearity condition which states that for every $x, y \in X$ exactly one of the following three conditions hold: (i) xRy , (ii) yRx or (iii) $x = y$ [4]. The ‘less than or equal’ relation denoted as ‘ $\leq$ ’ on the set of integers $\mathbb{N}$ is an example of a linear order relation since for all $x, y, z \in \mathbb{N}$ these four conditions hold:

1. Reflexivity ($x \leq x$).
2. Antisymmetry (if $x \leq y$ and $y \leq x$ then $x = y$).
3. Transitivity (if $x \leq y$ and $y \leq z$ then $x \leq z$).
4. Either $x \leq y$ or $y \leq x$ or $x = y$.

2.5 Hasse Graph

A Hasse graph is a simplified graphical representation that can be used to visualise a partial order relation. The main attribute of a Hasse graph is its ability to depict the ordering structure among the elements without including any unnecessary edges, resulting in a more economical and streamlined representation. The construction of the graph involves starting with a directed graph representation of the relation and then applying following three rules [5]:

1. Eliminate self-looping edges (i.e., reflexive pairs).
2. Eliminate edges implied by the transitivity property.
3. Omit the arrow heads from the remaining edges and arrange the elements such that if xRy then x is positioned below y .

For example, if we have the divisibility relation over the set $S = \{3,4,12,24\}$ represented as:
 $R = \{(3,3), (3,12), (3,24), (4,4), (4,12), (4,24), (12,12), (12,24), (24,24)\}$. We notice the relation is a partial order since it is reflexive, transitive, and antisymmetric. The directed graph representation of the relation is as follows:

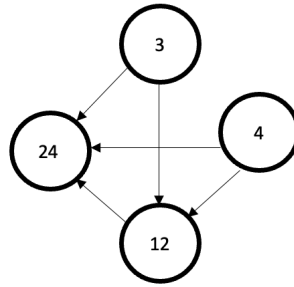


Figure 2.3: Directed Graph of Relation R

To construct the Hasse graph, we first eliminate the reflexive pairs that are indicated by bolded nodes.

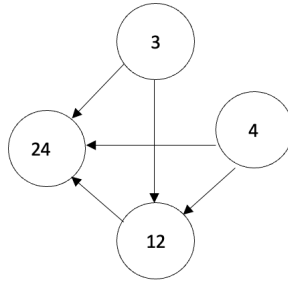


Figure 2.4: Graph after eliminating reflexive edges

We then eliminate all edges implied by the transitive property i.e (3,24) and (4,24).

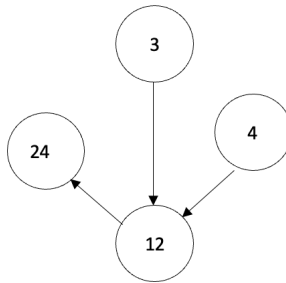


Figure 2.5: Graph After Eliminating Transitive Edges

Finally, we omit the arrow heads and reposition the nodes to indicate their relationships with each other.

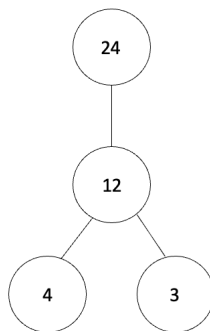


Figure 2.6: Hasse Graph of Relation R

2.6 The Warshall Algorithm

A transitive closure, denoted by R^* , on a binary relation R is defined as the smallest transitive relation containing R [6]. In order to find the transitive closure for a non-transitive relation, we use the Warshall's algorithm which computes the matrix of R^* . The transitive closure is computed through a series of $n \times n$ boolean matrices: $R(0), \dots, R(k-1), R(k), \dots, R(n)$ where n represent the number of elements in the relation's set [7]. The process of finding the transitive closure matrix can be defined by the following steps:

1. Start with matrix $R(0)$ which represent the original relation.
2. Generate a new matrix $R(k)$ following these two rules:
 - a. If an element in row i and column j is 1 in $R(k-1)$ then it remains 1 in $R(k)$.
 - b. If an element in row i and column j is 0 in $R(k-1)$ then it should be changed to 1 in $R(k)$ if the element in row i and column k and the element in row k and column j are both 1.

if the element in row i and column k and the element in row k and column j are both 1.

3. Continue from step two until $k = n$. This means we will have in total n rounds.

For example, if we have the following non-transitive relation R on the set $X = \{1, 2, 3\}$ where $R = \{(1, 1), (1, 2), (2, 3), (3, 2)\}$. The algorithm will have 3 rounds in total since the size of the relation's set = 3.

In the first round we will generate a new matrix $R(1)$ from the original matrix $R(0)$:

$$R(0) = \begin{array}{|c|c|c|c|} \hline & 1 & 1 & 0 \\ \hline 0 & 0 & 1 & \\ \hline 0 & 1 & 0 & \\ \hline \end{array} \longrightarrow R(1) = \begin{array}{|c|c|c|} \hline 1 & 1 & 0 \\ \hline 0 & 0 & 1 \\ \hline 0 & 1 & 0 \\ \hline \end{array}$$

As we can see, there is no change happened in this round.

In the second round:

$$R(1) = \begin{array}{|c|c|c|c|} \hline & 1 & 1 & 0 \\ \hline 0 & 0 & 1 & \\ \hline 0 & 1 & 0 & \\ \hline \end{array} \longrightarrow R(2) = \begin{array}{|c|c|c|} \hline 1 & 1 & \mathbf{1} \\ \hline 0 & 0 & 1 \\ \hline 0 & 1 & \mathbf{1} \\ \hline \end{array}$$

Notice here the element at the location (1,3) has changed to 1 since the elements at locations (1,2) and (2,3) both have a value of 1. Similar reasoning applies to the element at location (3,3). Refer to rule b from above.

In the third and last round:

$$R(2) = \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 0 & 0 & 1 \\ \hline 0 & 1 & 1 \\ \hline \end{array} \longrightarrow R(3) = \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 0 & \mathbf{1} & 1 \\ \hline 0 & 1 & 1 \\ \hline \end{array}$$

As we can see, the element at location (2,2) has changed to 1 since both the elements at locations (2,3) and (3,2) have a value of 1. Refer to rule b from above.

Therefore, the transitive closure for the relation R is:

$$R^{(*)} = \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 0 & 1 & 1 \\ \hline 0 & 1 & 1 \\ \hline \end{array}$$

$$R^{(*)} = \{(1,1), (1,2), (1,3), (2,2), (2,3), (3,2), (3,3)\}$$

Chapter 3

Literature Review

3.1 Brief

This chapter will highlight the importance of web-based interactive learning and its advantages on enhancing the quality of teaching. It will then introduce and critically analyse various interactive web-based systems used for teaching Discrete Mathematics, with a particular focus on systems that are relevant to the objectives of this project.

3.2 Web-based Interactive learning

Interactivity is essential for information acquisition and cognitive skill development [8]. With the outbreak of COVID-19 and the absence of interactive classrooms, there has been a growing literature on the design and development of interactive online platforms that can enhance the students' learning experience. As students nowadays are digital natives and heavily reliant on the internet, many educational institutions are working to build web-based learning systems (WBLs) that are aimed to improve content delivery. The advantages of WBLs lie with their ability to efficiently interact with the user [9]. However, in order to design an interactive and effective WBL, we need to consider a practical framework for building a powerful educational environment. One of the widely cited frameworks is the conversational framework introduced by Diana Lurillard's. The framework strongly suggests that learners must receive intrinsic feedback to reflect on their performance [10]. Interactive web-based systems are well-suited to adopt this framework since they can provide continuous feedback on user-initiated actions.

Inspired by the conversational framework, a three-way model of human-computer interactivity was proposed. This model consists of three main phases: computer initiation, learner response, and computer feedback [11].

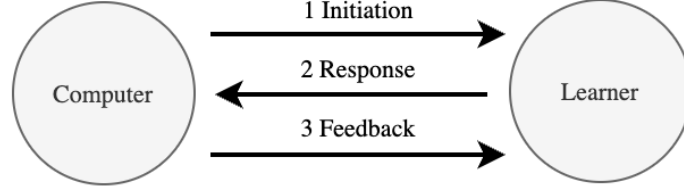


Figure 3.1: The three-way model applied to computer-initiated interactivity

The model can be used to describe several types of computer-initiated interactions. For example, in the case of our system, we present users with a set of checkboxes that corresponds to a pair, allowing them to create a relation by selecting these pairs. This constitutes the computer initiation phase. The learner response phase is initiated when users start checking or unchecking these boxes to establish a relation. Lastly, the computer feedback phase is initiated when the system present the properties that apply to the user’s established relation.

A growing body of research has been undertaken to evaluate the efficiency of this three-way model of interactivity in modern WBLs. A team in California state university has developed two versions of a WBL that teach colour theory: one non-interactive and lacking visual illustrations and an interactive version with dynamically manipulable visuals. The two versions were tested on 45 undergraduate students who were randomly assigned to either the interactive or non-interactive website. Students in both groups were then assessed on the principles they had learnt with a short quiz. The findings demonstrate that students in the interactive condition had higher mean telepresence scores and performed better on the quiz than students in the non-interactive condition [12]. Another similar study has developed an interactive and non-interactive versions of a system that teach the operation of a bicycle pump with the interactive version featuring pace control and an interactive simulation. Memory and problem-solving assessments were included to test the students understanding after they completed their learning. The results show that students using the interactive version had higher scores in the problem-solving test and spent less time to complete the memory test [13]. Hence, it is evident from the literature how the incorporation of interactivity as a design principle in a web-based learning system can significantly improve the learning quality of students.

3.3 Review of Existing Work

In this section, we will provide a comprehensive review on the various existing interactive tools and simulators that are relevant to the project. Our review will carefully examine their key features, capabilities and limitations. The following subsections will discuss these tools in detail.

3.3.1 Stanford's University Relation Editor Tool

[14] This relatively simple tool is developed by Stanford University as part of their *Mathematical Foundations of Computing* course. Users of the tool can experiment with binary relations, view a graphical representation of the relation, and get feedback on the properties that apply to the relation. The tool is reasonably interactive in that users can add and remove pairings from their relation using the provided checkboxes. However, the user freedom and control are limited. This is due to the relation's set being predefined as $\{a, b, c, d, e, f\}$ and being unable to be expanded, which leaves the user with few experimental options. Additionally, even though a relation can be represented in different ways, the tool's visualiser only shows the relation's graphical representation.

3.3.2 Graz's University Properties of Relations Tool

[15] This tool is developed by Graz University in collaboration with QHELP (Higher Education Learning Platform for Quantitative Thinking) initiative. Similar to the previous tool, users can create their own relation by adding or removing pairings from a list of provided checkboxes. The tool can then show certain properties of the relation and display an elegant Hasse graph if the relation is a partial order. Furthermore, users have the option to generate and experiment with random relations. This is a helpful feature as it allows users to get started with some ready-made relations before they construct their own relation. However, even though the tool offers many useful features, some limitations were noticed. Firstly, users are restricted to using the set $\{a, b, c, d, e\}$ rather than being able to customise and define their own set. Secondly, despite the tool's capability of showing which properties are satisfied by a given relation, the symmetry property is left unstated. Thirdly, it does not provide any feedback explaining why a certain property is not satisfied. Lastly, it does not offer a thorough feedback on the nature of the relation. For example, the relation can be successfully identified as a partial order when the conditions of a partial order relation are met, but it does not provide information on whether the relation is an equivalence or linear order.

3.3.3 Wolfram’s Demonstrations Relation Visualisation Tool

[16] This tool is developed by the Wolfram Demonstrations Project. It allows users to experiment with relations on the set $\{1, \dots, n\}$ where n represent the number of nodes to be created. Upon selecting the number of nodes, users can create their relation in a graphical form by adding directed edges between the nodes. The tool can display the relation in the form of ordered pairs or an adjacency matrix and it can show the different properties that are satisfied by the relation. Additionally, the tool provides the option of experimenting with hardwired relations such as “divides”, “congruent mod 3” and “ $a + 2b$ is prime”. However, the tool have some notable limitations. One such limitation is that it restricts users to relatively small sets, with maximum of 8 elements. This can constrain the users’ ability to fully explore the behavior of their relations on larger sets. Additionally, while users are able to create relations in the form of a directed graph, the tool does not provide the option to create relations as a matrix or ordered pairs.

3.3.4 Munich’s University Floyd-Warshall Algorithm Tool

[17] This tool is developed by Munich University to provide a step-by-step simulation of the Floyd-Warshall algorithm. The algorithm aims to find the shortest path between all pairs of vertices in a weighted graph. Users can begin the simulation by either uploading an existing graph, creating their own graph, or selecting one that is already hardwired into the system. The tool then runs the algorithm on the chosen graph and displays a description of the algorithm’s current state. To promote user engagement, the tool effectively integrates several interactive features such as “previous” and “next” control buttons, which enable users to navigate between the different steps of the algorithm at their own pace. It also provide a “fast-forward” option which runs the algorithm at an accelerated speed without the need to manually click “next” after each step. The tool also has a self-intuitive graphical user interface and pleasing visuals that signals the changes occurring at each step of the algorithm’s execution.

3.3.5 San Francisco’s University Floyd-Warshall Algorithm Tool

[18] This is another step-by-step simulator tool developed by San Francisco University to demonstrate the Floyd-Warshall algorithm. Unlike the previous tool, users are constrained to using randomly generated graphs and are unable create their own graph. Despite this limitation, the simulator excels in its interactivity, offering users the flexibility to traverse between the different steps of the algorithm at their own pace or to play an animation of the algorithm at a personalised speed. The tool also features interactive visuals that can effectively highlight any significant changes taking place during each step of the execution. The interactive elements utilised in this tool were adapted and repurposed to create our Warshall simulator tool.

3.4 Extending Existing Tools: Enhancing Capabilities and Overcoming Limitations

This project aims to build on the existing interactive binary relation tools by expanding on their capabilities and overcoming their key limitations. As discussed in the previous sections, the current binary relation tools exhibit certain shortcomings that hinders the user’s learning experience. One of the main limitations of these tools is how rigidly they define the relation’s set, which is restricted to predefined letters of the alphabet or single numeric values. Our system overcomes this limitation by allowing users to define their own relation set without any restriction on its content. The system also permits users to flexibly create sets consisting of any combination of numerical and alphabetical values. Additionally, the size of the relation’s set in the existing tools is relatively small, which is limited to a maximum of 8 elements. The project overcomes this limitation by allowing users to experiment with large sets of up to 50 elements.

Moreover, the current tools are lacking contextual support that can help provide a guided learning. Drawing inspiration from the conversational framework, the project overcomes this limitation by providing the user with an intrinsic feedback to illustrate, for example, why a certain property is not satisfied; thus offering a more informed learning experience. The project also extends on the current tools by providing users with the ability to save and delete their relations, giving them greater flexibility and control. Furthermore, the system expands on the functionality of the existing tools by enabling users to create their desired relation in any representation of their choice while allowing them to view the alternative representations. Lastly,

it is worth noting that even though we have simulator systems that demonstrate the Floyd-Warshall algorithm on graphs, we are lacking similar systems that demonstrate the Warshall algorithm on binary relations. To address this gap, the project has incorporated features from existing simulator tools to create a simulator tool that is tailored to our needs. Our simulator has also been extended to provide the learner with an option of generating a PDF summary of the algorithm's execution.

To summarise, our system enhances the existing interactive binary relation tools by enabling users to define their own relation set, experiment with large sets, receive contextual feedback, and have greater control over their relations. The project also addresses the lack of simulator tools for demonstrating the Warshall algorithm on binary relations. These expanded capabilities and features help us offer a flexible and personalised learning experience, making the project a valuable asset for students and educators.

Chapter 4

Requirements & Specification

4.1 Brief

This chapter will begin by introducing the functional requirements that must be met to achieve the project's objectives. Following that, it will outline the non-functional requirements which highlights the qualities and characteristics of the system. The chapter then moves on to discuss the framework used to build the website and the development details for the frontend and backend. The chapter will then conclude with the specification of our three interactive learning tools.

4.2 Functional Requirements

The software functional requirements will be split into two parts: user and system requirements. These requirements will explain the needs from the standpoint of the user and the system respectively. Each of these requirements will be associated with a tag indicating its category. The tags are defined as follows:

$\langle UA \rangle$: User Account

$\langle RV \rangle$: Relation Visualiser Tool

$\langle WS \rangle$: Warshall Simulator Tool

$\langle RE \rangle$: Relation Explorer Tool

4.2.1 User Requirements

Requirement ID	Requirement Description	Requirement Tag
U.1	The user must be able to register in the system	<UA>
U.2	The user must be able to login to their account	<UA>
U.3	The user must be able to reset their password	<UA>
U.4	The user must be able to log out from their account	<UA>
U.5	The user must be able to create a relation	<RV> <WS> <RE>
U.6	The user must be able to save a relation	
U.7	The user must be able to load a previously saved relation	
U.8	The user must be able to delete a relation	
U.9	The user must be able to clear a relation	
U.10	The user must be able to apply search keywords to filter their saved relations	
U.11	The user must be able to create their relation as a directed graph and view the alternative representations of the relation.	<RV>
U.12	The user must be able to create their relation as a matrix and view the alternative representations of the relation.	<RV>
U.13	The user must be able to create their relation as ordered pairs and view the alternative representations of the relation.	<RV>
U.14	The user must be able to execute the Warshall algorithm on their relation	<WS>

U.15	The user must be able to navigate back and forth between the different steps of the algorithm.	<WS>
U.16	The user must be able to run the Warshall algorithm in fast forward mode	<WS>
U.17	The user must be able to adjust the speed of the algorithm's execution	<WS>
U.18	The user must be able to save a PDF file that summarizes the algorithm's execution	<WS>
U.19	The user must be able to save the algorithm's final result (i.e. transitive closure relation)	<WS>
U.20	The user must be able to see the properties that apply to their relation (i.e. reflexive, transitive, symmetric or antisymmetric)	<RE>
U.21	The user must be able to see the types that apply to their relation (i.e. equivalence, partial, or linear order)	<RE>
U.22	The user must be able to enforce a property on their relation	<RE>
U.22	The user must be able to see a Hasse graph of the relation if it is a partial order	<RE>
U.23	The user must be able to switch between the Hasse graph and the Directed graph of their relation	<RE>
U.24	The user must be able to display the Directed/Hasse graph in full screen mode	<RV, RE>
U.25	The user must be able to save the Directed/Hasse graph in PNG format	<RV, RE>

Table 4.1: List of the User Requirements

4.2.2 System Requirements

Requirement ID	Requirement Description	Requirement Tag
S.1	The system must validate the user credentials	<UA>
S.2	The system must update the user password when a reset request is initiated	<UA>
S.3	The system must process and validate the user input	<RV> <WS> <RE>
S.4	The system must create a new record in the database for the relation when a save request is initiated	
S.5	The system must enforce a unique constraint on the stored relations. If the user attempts to save the same relation over the same set, the system should reject the request	
S.6	The system must delete the relation selected by the user from the database when a delete request is initiated	
S.7	The system must be able to retrieve all the relations saved by the logged in user	
S.8	The system must be able to clear the state of the relation when a clear request is initiated	
S.9	The system must allow users to visualise their relation in a directed graph, matrix or ordered pairs.	<RV>
S.10	The system must update the alternative relation representations in real time whenever the user alters their relation.	<RV>
S.11	The system must run the Warshall algorithm on the user relation	<WS>
S.12	The system must show the results of the algorithm in a step-by-step fashion	<WS>
S.13	The system must highlight the cells that has changed within each step of the algorithm's execution	<WS>

S.14	The system must generate an explanation text for each step of the algorithm's execution	<WS>
S.15	The system must handle user events (i.e. clicking next or previous) and respond accordingly	<WS>
S.16	The system must generate a PDF file containing a summary of the algorithm's execution in a reasonable format.	<WS>
S.17	The system must check the constraints of the reflexive, symmetric, antisymmetric and transitive properties against the user relation and show if any of them are satisfied	<RE>
S.18	The system must check the constraints of the equivalence, partial and linear order against the user relation and show if any of them are satisfied	<RE>
S.19	The system should perform a search for counterexamples and display them to the user as a justification for why a particular property or type is unsatisfied	<RE>
S.20	The system must update the state of the relation properties and types when the user alters the relation	<RE>
S.21	The system must construct and show a Hasse graph if the relation is being identified as a partial order	<RE>
S.22	The system must modify the state of the Hasse graph in real time whenever the user alters their relation	<RE>
S.23	The system must allow the user to generate random relations	<RE>
S.24	The system must be able to export the Directed/Hasse graph to PNG	<RE> <RV>

Table 4.2: List of the System Requirements

4.3 Non-Functional Requirements

This section will outline the system's non-functional requirements. These requirements define some attributes which are necessary to ensure that our system meets the highest quality standards. These attributes include factors such as security, usability, compatibility, performance, accessibility, scalability, maintainability and documentation.

Requirement ID	Requirement Description	Requirement Category
N.1	The system should use a safe password hash.	Security
N.2	The system should be user-friendly and self-intuitive	Usability
N.3	The system should be compatible with different operating systems and web-browsers.	Compatibility
N.4	The system should respond to user interactions in real time.	Performance
N.5	Each page should load within a maximum of 3 seconds.	Performance
N.6	The system should be accessible to a wide range of users including people with vision deficiencies.	Accessibility
N.7	The system should be scalable such that it can be easily extended with more learning tools.	Scalability
N.8	Each learning tool should be decomposed into separate components that handle well-defined tasks.	Maintainability
N.9	The source code should be consistent and well-documented with class and method comments..	Documentation

Table 4.3: List of the Non-Functional Requirements

4.4 Specification

4.4.1 Server-side Framework

The website has been developed using Django, a framework for building web applications with Python. The reason for choosing this framework is because it follows a Model-View-Template (MVT) architecture which allows modifying the UI design of a web-application separately from the business logic. This is particularly important in this project since we need to separate the development of the visual illustrations and interactive components from the program logic. Another reason for choosing Django is because it is a very solid framework with many built-in security features including password hashing. With such security feature offered by Django it was feasible to fulfil our security requirement N.1. Django has also an Object Relational Mapper (ORM) which is able to communicate and interact with database engines like MySQL. This is useful in our project since we require a database for saving and retrieving relations. Django also offers a cross-platform development which will allow the system to run on multiple operating systems. Overall, the decision to use Django was based on its wide range of features and its robust architecture.

4.4.2 Frontend

The frontend was developed using mainly JavaScript which is the leader in interactive web-development. JavaScript is a very light language that encourages interactivity by allowing the creation of dynamic and interactive webpages which can effectively engage the end-users. JavaScript also comes with a wide range of visualisation and interactive-based libraries that are useful for our project. In addition, using a lightweight language like JavaScript enabled us to optimise the application execution speed which is a critical element to ensure the interactivity and responsiveness of our system. Alongside JavaScript, Hypertext Markup Language (HTML) was utilised to define the structure of the pages and Cascading Style Sheets (CSS) to control their style, presentation, and layout.

4.4.3 Backend

The backend was mainly developed using JavaScript. By using JavaScript, we allow easy communication between the frontend and backend as they are both developed using the same language. In addition, we have also used Python which is required by Django to build some essential components such as the website's forms and URLs. The combination of these two languages allowed for efficient communication between the different parts of the system and ensured compatibility with our chosen framework.

4.4.4 Learning Tools

Relation Visualiser

This tool offers multiple representations of a binary relation, allowing users to visualise and manipulate their relation in a directed graph, matrix or ordered pairs. Users can create their relation in a graphical form, where the system generates a set of labelled nodes based on the elements in the set. The user can then create their relation by adding directed edges between the nodes. During the creation of the relation, the tool will display the matrix and ordered pairs representations, which will be updated in real time when the directed graph is modified. Alternatively, users can enter their relation in a matrix form, where the system creates a matrix in the form of a table with row and column headers corresponding to the elements in the set. Users can click the table cells to toggle their value from 0 to 1 or vice versa to indicate the presence or absence of a pair in the relation. The tool will then display the graph and ordered pairs representations, which will be updated in real time whenever a cell is clicked. For those who prefer to work with ordered pairs, the system will generate a set of checkboxes of all possible pair combinations based on the set elements. Users can add or remove pairs from their relation by checking or unchecking these boxes and the tool will update the matrix and directed graph representations accordingly. Overall, the tool helps learners in understanding binary relations by offering them multiple representations to visualise and manipulate their relations.

Warshall Simulator

This tool provides users with a comprehensive understanding of the Warshall algorithm to find the transitive closure for a relation. It offers a simulation of the algorithm in a step-by-step fashion, allowing users to observe the changes occurring on the matrix at each step. To be able to initiate the simulation, users should input a relation of their choice. Following that, the system will generate n matrices, where n corresponds to the size of the relation's set. Each of the generated matrices maps to one round of the algorithm's execution. These matrices will not be displayed to the user in one-go but will require the user to navigate between them at their own pace, promoting their continuous interaction and engagement. The tool also offers a feature of saving the resulting transitive closure relation. In addition, it allows users to generate a PDF document with a summary of the algorithm's execution. This feature provides users with a tangible outcome of the simulation that can be used as a study reference. Overall, the tool is designed to assist learners in understanding the process of finding the transitive closure for a binary relation.

Relation Explorer

The primary objective of this tool is to enable users experiment with various relations and obtain a real-time feedback on the properties that apply to their relations. Based on these properties, the relation will be further classified to different types such as equivalence, partial, or linear order. The tool is equipped to track any modifications to the relation, including the addition or removal of pairs, and update the graphical user interface to reflect the state of the relation's properties and the types that apply. If a property or a type is not satisfied by the relation, the tool provides users with contextual support by showing counterexamples in the form of tool-tips. In addition, when the relation satisfies the partial order constraints, the tool displays its corresponding Hasse graph which can be optionally exported as a PNG. Users also have the option to switch between the Hasse graph and the original directed graph of the relation. This option allows the user to analyse and compare the differences that resulted from transforming the directed graph into the simplified Hasse graph. Overall, this tool empowers learners to interactively experiment with binary relations to gain a deep understanding of their different types and properties.

Chapter 5

Design

5.1 Brief

This chapter will start by showcasing some illustrative diagrams that are aimed to reveal the system design at an abstract level. It will then elaborate upon the rationale underlying some design decisions. Following that, the chapter will introduce the design principles of the graphical user interface (GUI), the interactive elements used, colour scheme and overall aesthetics of the website. Finally, it will conclude with the system’s accessibility and responsiveness.

5.2 System Architecture

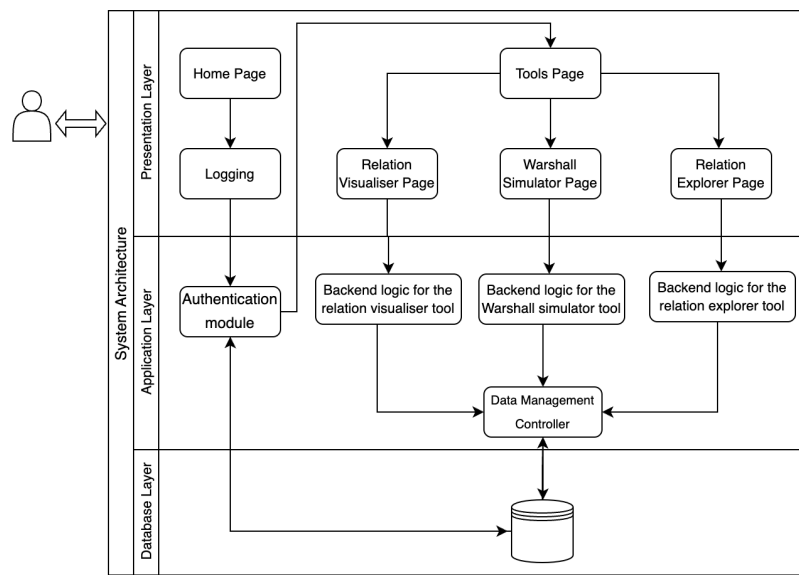


Figure 5.1: System Architecture Diagram: Layers and Components

Our website follows a system architecture of three layers: presentation, application, and database. The presentation layer provides the user with the interface components for accessing the learning tools. The application layer, on the other hand, contains the authentication module which utilises Django’s built-in authentication capabilities to handle access control. This module communicates with the database to verify the user credentials and grant them access to our learning tools upon successful authentication. Additionally, this layer contains the business logic and the implementation of the different tools offered by the system. It also includes the data management controller that acts as an intermediary between the user and the database. In the context of the Django framework, the controller represent a Django view which is a special python function that processes HTTP requests and coordinates with the database to manage user actions such as saving, deleting or retrieving relations. Lastly, the database layer is the central repository of information that stores data related to the users registered in the system and their relations.

5.3 Use Cases

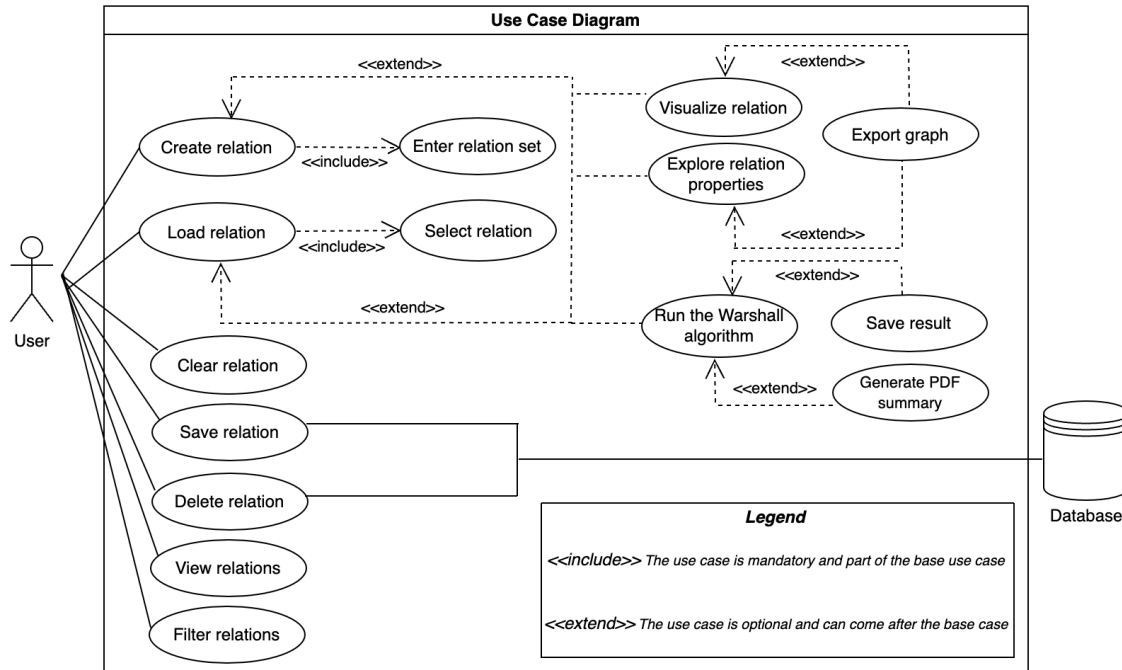


Figure 5.2: The System’s Use Case Diagram

The use case diagram captures the actions a user can take to interact with our system. As shown in the diagram above, creating a new relation requires the user to specify a valid set, while loading a relation requires the selection of a previously saved relation, if one exists. Once a relation is available, the user can visualise it through various representations using our visualiser tool or they can alternatively choose to explore the relation properties through our relation explorer tool. In the event that either of these actions is performed, the user can optionally export the generated Directed or Hasse graph as a PNG file. If the user decides to run the Warshall algorithm using our Warshall simulator tool, they can choose to save the resulting transitive closure of the algorithm or generate a PDF document with a full summary of the algorithm's execution. In addition to the above use cases, the user can manage their relations by saving or deleting them, which will consequently trigger the database to retrieve and process the relevant data. Lastly, the user can clear the state of their relation or search and filter through their relations as needed.

5.4 Function-Based Design: A Modular Approach

The system has been designed using a function-based programming approach. In a function-based programming, the system heavily relies on the composition and communication between functions rather than class objects. The design decision to use function-based programming was driven primarily by the fact that our system did not require the instantiation of multiple instances of a class object with shared state and behaviors nor it requires the use of a hierarchical structure such as class-inheritance. In addition, as the platform emphasises interactivity and engagement, using function-based programming is well-suited for our system since functions tend to be lighter weight than classes, enabling us to optimise page loads and ensure a smoother learning experience. Therefore, to meet the functional requirements of the project without adding unnecessary complexity to the code-base, we grouped related functions into separate script files. These functions are standalone and can be invoked independently from one another but communicate together to accomplish more sophisticated tasks. By adopting this approach, we allow for a streamlined development process as each function can be designed independently and can be easily integrated with the overall system. Therefore, this approach created a modular design architecture which enhances flexibility, efficiency and code maintenance.

5.5 Software Design Pattern

The system has been developed following the Model-View-Template (MVT) design pattern. In this design pattern, the Template handles the presentation layer which represents the user interface (UI), the Model handles the retrieval and management of data, whereas the View handles the logic of the application [19]. In our project, we follow this design pattern by separating the HTML files that contain the UI elements (Template) from the JavaScript and Python files which represent the data management layer and application logic (Model/View). This separation of concerns helped achieve a more modular and maintainable architecture by making it easier to modify the business logic of the system without affecting its interface.

5.6 Data Flow

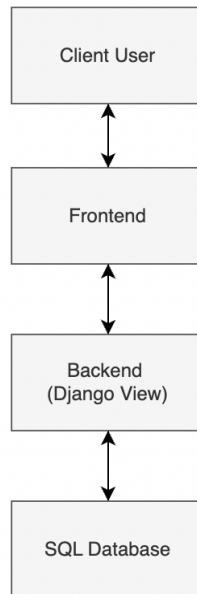


Figure 5.3: Data Flow Diagram

The diagram above shows the high-level flow of the data in our system. It starts with the client user who interacts and initiate requests to the frontend. Consequently, the frontend will handle the requests initiated by the user and send any associated data to the backend server. In the backend, a Django view will receive the data and communicate with our SQL database to perform the necessary operations such as saving or deleting a relation. The database will then be responsible to process and manage these queries.

This data flow architecture provides a separation of concerns between the frontend, backend and the database, allowing each of these components to operate independently while still communicating with one another to allow a dynamic and easy data flow.

5.7 Data Management

As discussed in our system’s requirements, users should be able to save binary relations as part of their learning process. Even though there are various approaches to achieve this functionality such as using XML or other data storage formats, the project uses Django’s built-in data models to satisfy this requirement. Django models act as Python objects which map to a table in the SQL database. These models enable us to perform efficient database operations such as saving or deleting relations associated with users. The decision to use Django models was a deliberate design choice as these models provide a standardised way of managing data with a built-in support for data migrations. In our system, we utilise two primary models. The first one is the **user model** which stores data related to a registered user and the second one is the **relation model** which stores data related to a binary relation.

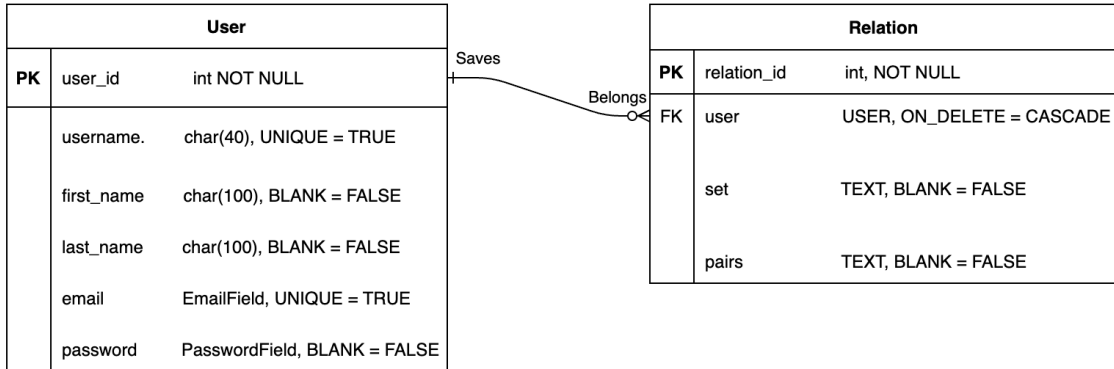


Figure 5.4: Entity Relation Diagram (ERD) for the System Models

As seen from the diagram above, the relation model has a foreign key, “user”, that links each record in the relation model to a single record in the user model. The relationship between the user model and the relation model is one-to-many because a user can save multiple relations but a single relation record belongs to one user. The relation model comprises three fields: “set”, representing the set of the relation, “pairs” representing the ordered pairs of the relation and “user” which acts as a foreign key. When combined, these three fields form a unique constraint which ensures that a user cannot save the same relation over the same set more than once. However, the constraint allows the user to save identical relations if they originate from

different sets. Enforcing this constraint is a key design decision for our database schema. The reason behind this design choice is to allow the user to access and modify the same relation over different sets. For instance, let's examine the following scenario where the user creates the relation $R = \{(1,1), (2,2), (3,3)\}$ over the set $\{1,2,3\}$ and subsequently creates the same relation over the set $\{1,2,3,4,5\}$. By storing both relations in the database over these different sets, the user can add additional pairs to the second relation such as $(4,4)$ or $(5,4)$, which contain set elements not present in the first set. Hence, enforcing this constraint not only considers the relation itself but also the set from which the relation originates, allowing users to maximise the usefulness of their saved relations.

5.8 Graphical User Interface

5.8.1 Design Principles

To enhance the quality of the frontend design, It is critical to integrate some essential GUI design principles. These design principles contribute to the interface being intuitive, simple to use, and visually appealing. They also assist to provide a consistent user experience across different platforms and devices. The project considered several GUI design principles, which are indicated below with examples of how they were applied.

Consistency

The consistency design principle indicates that the user interface should be consistent in terms of look and feel, navigation, and interaction [20]. In line with this principle, the project has unified the style and overall design of our three learning tools. This includes background colour, size of buttons, text font and positioning of elements. Consistency has also been considered when modeling reflexive nodes in a directed graph. As the project utilises two different libraries to visualise directed graphs, we have noticed inconsistencies in the representation of reflexive pairs across the two libraries. One library signals reflexivity by nodes with self-looping edges while the other library uses nodes with bolded borders. Therefore, in order to avoid confusion and ensure a consistent representation to the user, the project has unified the representation of reflexive pairs in a directed graph to nodes with bolded borders.

Feedback

The feedback design principle states that users should be provided with feedback when they interact with the interface [20]. The project incorporates this principle in various ways. For instance, the system displays error or success messages with appropriate colour cues based on the outcomes of the user actions.

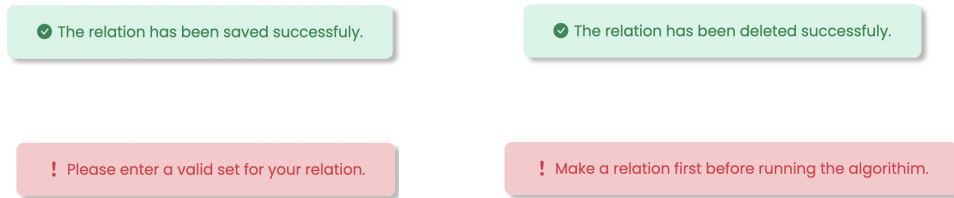


Figure 5.5: Examples of Success and Error Messages Generated by the System

The system also uses pop-up windows which act as a guide to inform users on the consequences of their actions, such as when attempting to delete a relation or switching between tabs while running the Warshall algorithm. With these pop-up windows, we help users prevent unintended actions or accidental errors.

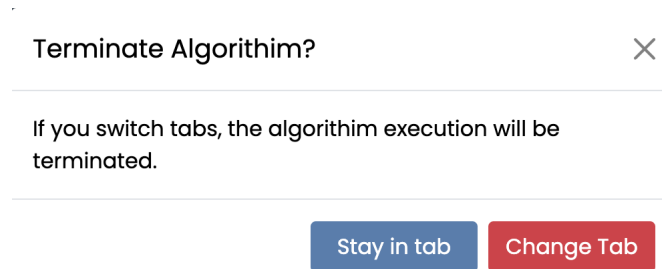


Figure 5.6: Warning Pop-up Window when Switching Tabs during the Algorithm Execution

Finally, feedback is also integrated across the site in the form of tool-tips which provide explanations for potentially ambiguous elements. With these guided tool-tips, users can understand the features and content of the website regardless of their level of experience.

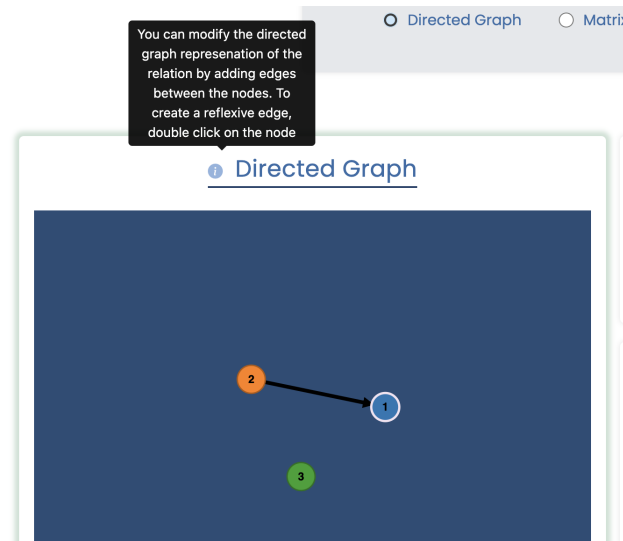


Figure 5.7: Tooltip: Guide to Interacting with the D3 Graph

Error Prevention

The error prevention design principle states that the user interface should be designed in such a way that it minimises the chances of errors [20]. To reduce the chances of users making mistakes when entering their relation, user input is kept to a minimum. For instance, when users choose to create their relation in an ordered pairs form, they will not have to type it out, as this can lead to errors such as missing commas or parentheses. Instead, the system will generate all possible pair combinations from the relation's set and the user can then select from these pairs to create a relation. Similarly, when users create their relation in a matrix form, they will not be asked to type a number in each cell but instead they can simply click on the cells to toggle their value from 1 to 0 or vice versa. These design decisions do not only prevent errors but also minimize the effort required from the user.

Recovery

The recovery design principle states that users should be able to easily undo or reverse any action they have taken [20]. The project has taken this design principle into account by allowing users to easily modify their relation by removing pairs they have added, delete a relation they have saved, or clear any relation that they have created and start over.

5.8.2 Interactive Elements

Our system features a range of interactive elements that have been carefully integrated to enhance the user's learning experience. For example, the system has interactive GIFs which provide feedback based on the user actions, a table with togglable cell values, checkboxes to help construct a relation and an interactive graph that allow users to connect nodes with edges and experiment with different relation configurations. Additionally, using the three-way model applied to computer-initiated interactivity as a guiding principle, we tried to increase the user interactions with the website in several ways. For example, we offer a walk-through of the Warshall algorithm accompanied with some explanations. This walk-through example is not presented to the user all at once but requires their active participation to progress through the example by clicking on the “continue” button.

What's the Warshall's Algorithm?

The Warshall's algorithm computes the transitive closure R^* of a binary relation R . The transitive closure is computed through a series of $n \times n$ boolean matrices where n represent the number of elements in the relation set.

The process of finding the transitive closure matrix using the Warshall's algorithm can be defined by the following steps:

1. Start with matrix $R(0)$ which represent the original relation.
2. Generate a new matrix $R(k)$ following these two rules:
 - a. If an element at position (i,j) is 1 in $R(k-1)$ then it remains 1 in $R(k)$.
 - b. If an element at position (i,j) is 0 in $R(k)$ then it should change to 1 if the elements at positions (i,k) and (k,j) are both 1 in $R(k-1)$.
3. Continue from step (2) until $k = n$.

Let R be a relation on $\{a, b, c, d\}$. $R = \{ (a,d), (b,a), (b,c), (c,a), (c,d), (d,c) \}$. Let's assume the listing order is: a,b,c,d .

Original Matrix

☐	☐	☐	☐
☐	☐	☐	☐
☐	☐	☐	☐
☐	☐	☐	☐

→ Continue



Figure 5.8: Illustration of the Warshall Algorithm by a Walk-through Example

Moreover, to assess the user's comprehension before they run the algorithm on their own relations, we included an interactive drag and drop test question. By including this type of interactive assessment, we ensure users' engagement and help them solidify their understanding of the algorithm.

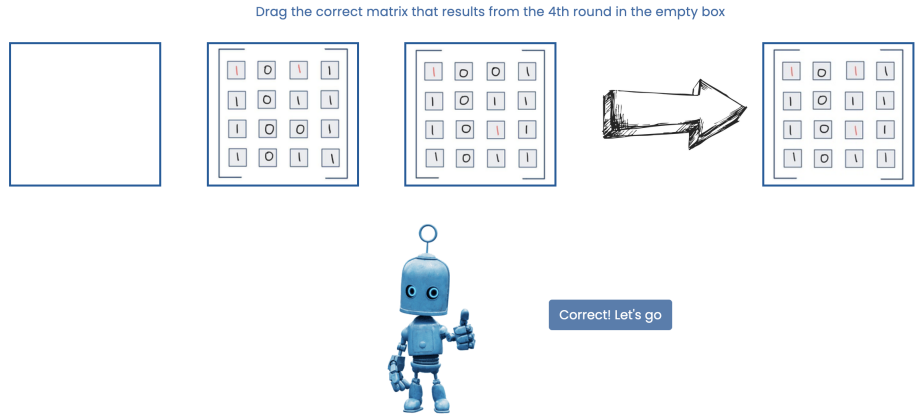


Figure 5.9: Interactive Drag and Drop Question

Furthermore, we recognize the benefits of interactive visualisation in enhancing the learning experience. Therefore, we have added indicator icons adjacent to each relation type. These icons are interactive and light up selectively only when the corresponding type applies to the relation being analysed while remaining off when it does not. By means of this interactive visualisation, users can easily identify the nature of their relation.

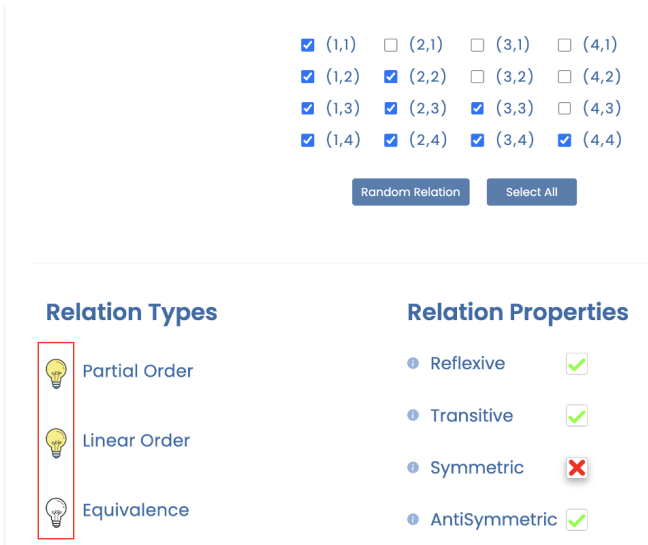


Figure 5.10: Visual Representation of the Relation Types

In addition, we have integrated interactive tool-tips that are displayed when the user hovers their mouse over certain elements. Such tool-tips provide users with contextual support that can improve their understanding of the material.

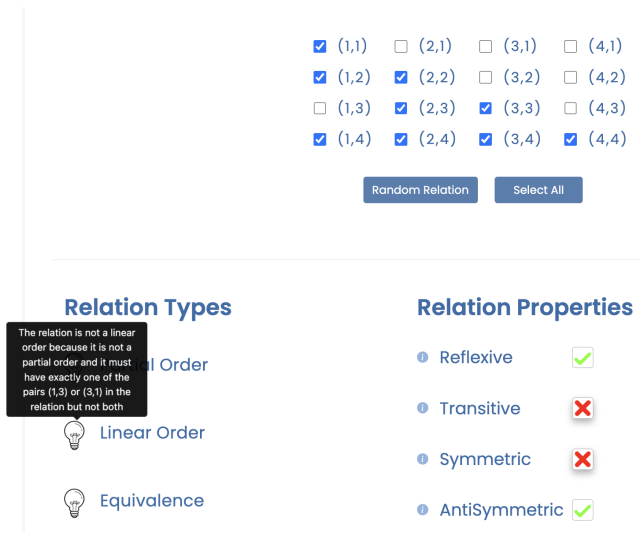


Figure 5.11: Counterexample Tool-tip for a Non-linear Relation

Overall, by incorporating these interactive elements, we aim to enhance the learners’ engagement with the topic and help them gain better understanding of binary relations and their applications in a variety of contexts.

5.8.3 Aesthetics and colour Scheme

Given that our website functions as a learning platform, it is crucial to consider the aesthetics and colour scheme of the graphical user interface. Research studies have emphasised the importance of a clean design in helping learners process the material more fluently, leading to better learning outcomes [21]. Bearing this in mind, we have placed a strong emphasis on designing a visually appealing website with a carefully chosen colour scheme of blue and yellow. This colour choice was based primarily on a study that compared the effects of 12 different colour combinations on the participants’ visual performance and found that the combination of blue and yellow has led to the best performance [22]. Another study assessed the relationship between the trust of a website with four colour schemes (red, green, black and blue) and found that blue was perceived as the most trustworthy colour scheme [23]. In addition to carefully choosing the colour scheme, we also integrated animated and colourful GIFs instead of static images which added a dynamic and lively feel to the pages and further enhanced the overall aesthetics of the website.

5.8.4 Accessibility

Full-Screen Graph Display

To ensure that our system is accessible to many users including those with vision deficiencies, we added a feature that enables users to display the graphs in full-screen mode. This feature allows users to enlarge the graph and avoid distractions from the surrounding page elements. With the addition of this feature, we seek to make the system accommodates the preference and needs of different users.

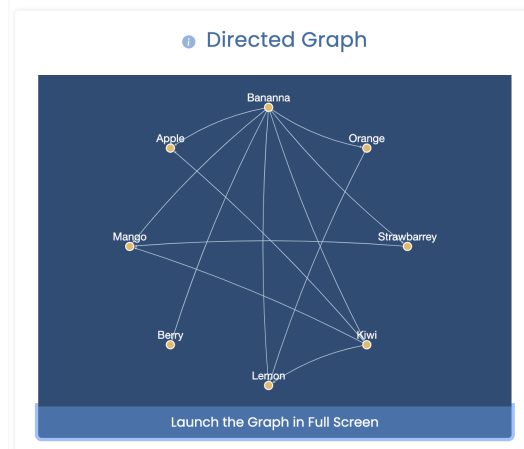


Figure 5.12: Display of the graph without full-screen mode

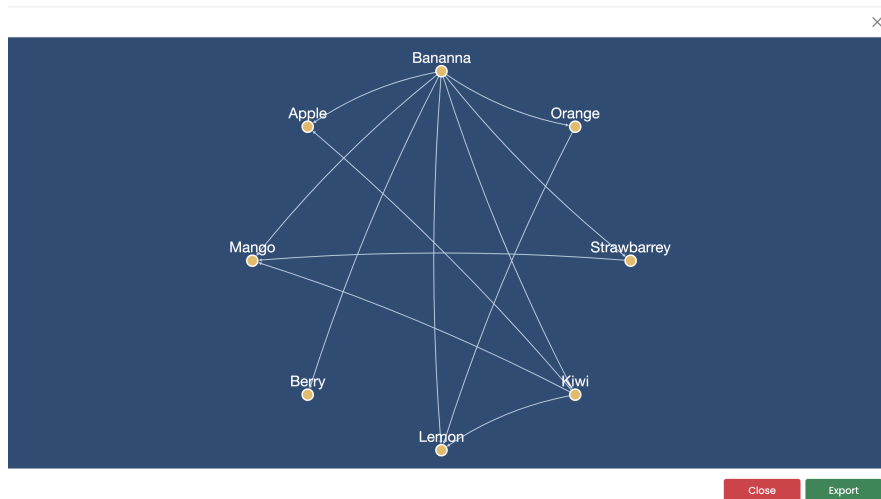


Figure 5.13: Display of the graph with full-screen mode enabled

Responsiveness

In our pursuit of providing an accessible and inclusive learning platform, the system has been designed to be responsive to different screen sizes. This has been achieved by first detecting the size of the screen being used with the help of some CSS properties and adjust the elements on the page accordingly. This responsive design allows the website's content to remain clear and easily accessible from a range of different devices. As shown in the figures below, the elements in the page have been rearranged to appropriately fit the screen size. For instance, on tablet screens, we adjusted the position of the speed slider and navigation controls. On smartphones, the box is positioned below the matrix, removing the need for horizontal scrolling and ensuring that all the content is visible and easy to access.

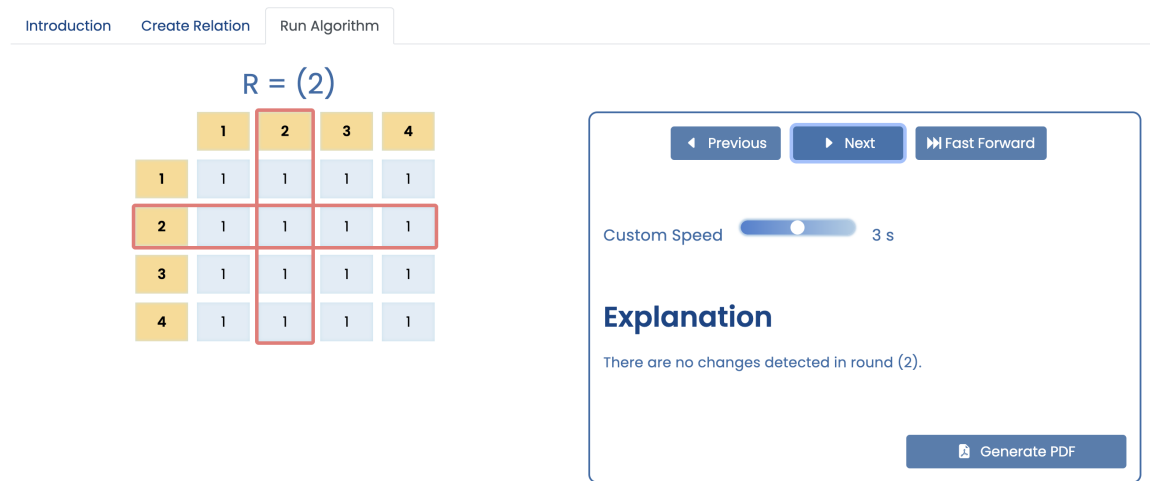


Figure 5.14: Warshall Simulator Tool Interface on Laptop Screen

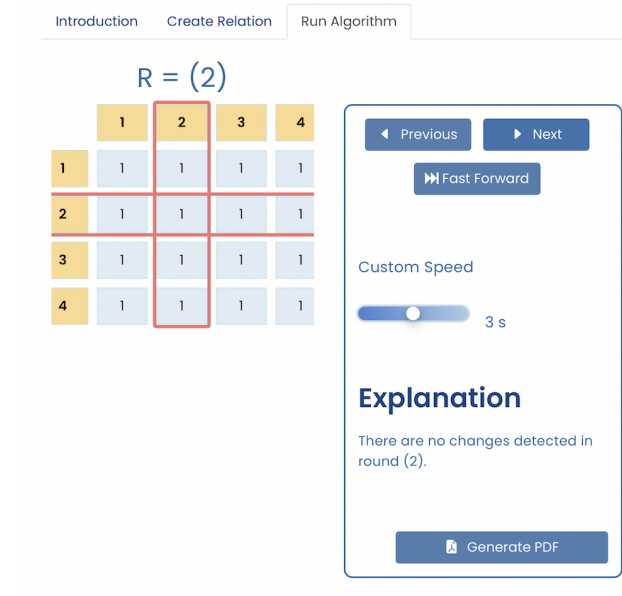
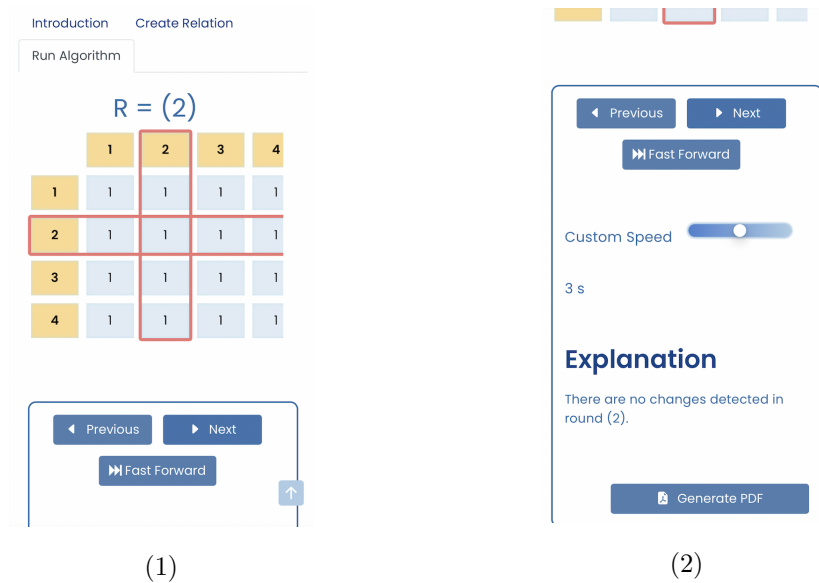


Figure 5.15: Warshall Simulator Tool Interface on Tablet Screen



(1)

(2)

Figure 5.16: Warshall Simulator Tool Interface on Smartphone Screen

Chapter 6

Implementation

6.1 Brief

This chapter reveals how the project requirements were implemented. It begins by describing the development approach, followed by a list of all third-party libraries that have been utilised. Following that, the chapter will provide a description of the general code structure. Next, it will cover how relations are saved, deleted, retrieved and parsed. Lastly, the chapter will explain how each of our three learning tools has been implemented.

6.2 Development Approach

The project has taken an iterative development approach to achieve the functionalities of the system. Prior to the implementation stage, the system requirements were defined and structured into different categories as outlined in the Requirements and Specification chapter. Following that, multiple development sprints ¹ have been taken where each sprint involves implementing a subset of the system requirements based on their priorities and dependencies. After each sprint, the implementation was checked for validity and has been refactored for maintainability. This process was repeated until every requirement of the system was achieved. In addition, it is worth noting that even though refactoring usually comes at a later stage of the development process, carrying refactoring after each sprint has helped building up the code on a clean base and avoiding technical debt ².

¹Sprint: Repeatable short-time period to complete a set of the system's functionalities [24].

²Technical debt: The cost implied to refactor the code as a result of considering inefficient solutions [25].

6.2.1 Third-Party Content

The project has utilised some external libraries, plugins and frameworks which helped fulfilling the system's requirements and reduced the project's development time. Below, we will provide a brief description on each one of them.

D3.js

D3.js [26] is an open-source JavaScript library to visualise and manipulate data. It provides a framework for building dynamic visualisations. The D3-force directed graph API was used in this project to allow users create and manipulate directed graphs.

CytoScape

CytoScape [27] is an open-source JavaScript library which is used to model, visualise, and analyse networks as well as graphs. It is compatible with all modern browsers making it a good choice for our project. The library was used to display the directed and Hasse graph of a relation.

jQuery

jQuery [28] is a lightweight and open-source JavaScript library which was used in this project to enable us to communicate and exchange data with the Django server.

jsPDF

jsPDF [29] is an open-source JavaScript library for generating PDF documents. This library is useful for our Warshall simulator tool as it allowed us to satisfy requirement S.16.

Bootstrap

Bootstrap [30] is an open-source frontend framework which facilitate creating the elements of the user interface (UI) such as buttons, forms and menus. The project has used Bootstrap to build the frontend components, ensuring a modern and contemporary appearance of the pages.

qTip2

qTip2 [31] is a jQuery plugin that allows creating customisable tool-tips and pop-ups. In this project, we integrated the use of qTip2 and Cytoscape.js extension in order to display tool-tips associated with the nodes generated by the CytoScape library.

6.3 Code Structure

The project has organised the script files that define the implementation of the system into multiple packages. This separation of packages helped group related files together, making it easier to navigate through the code-base. Within each file, the code has been further organised into smaller components by defining functions that carry a well-defined task. The project organises all of these script files into a package called “js_files”. This package contains several sub-packages. For instance, the “ajax” sub-package holds the files related to making AJAX requests, while the “app” sub-package contains further sub-packages that have the necessary components for achieving the core functionalities of the system. Inside the “app” package we have the “generators” sub-package that includes files which can generate the different relation representations. The “relation_operations” sub-package includes files that handle formatting and parsing relations, while the “shared” sub-package contains the shared variables between our three tools. In addition, there is a “UI” sub-package that holds the files which handles the user interactions with the interface and the “validation” sub-package which contains files that validates user inputs and selections.

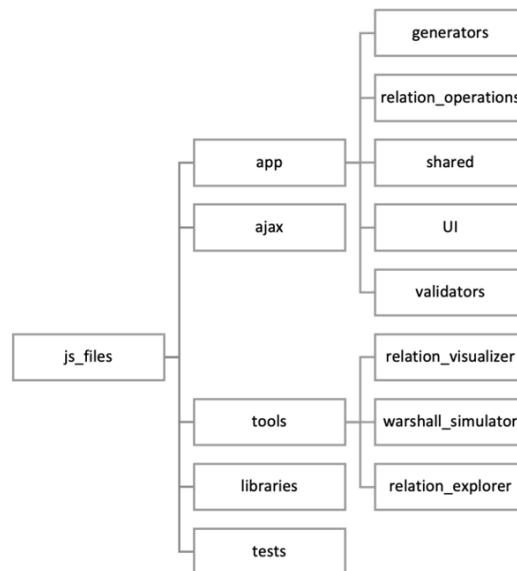


Figure 6.1: Directory Structure for the Main Backend Components of the System

The tools of our system are implemented in the “tools” package. These tools have been structured consistently, each having a “controller” package with two main files: “runner.js”, which contains the core functions to run the tool and “state.js” which defines the state of the variables that are shared within the tool. Overall, the code structure of the system is well-organised and follows common software development practices for keeping a maintainable code-base.

6.4 Saving, Deleting and Retrieving Relations

Saving, deleting and retrieving relations is achieved through the use of AJAX (Asynchronous JavaScript and XML) which is an approach used in web-development to exchange data between a client and a server [32]. In this project, we use AJAX requests to communicate between the frontend and backend, providing access to the SQL database where our Django models resides.

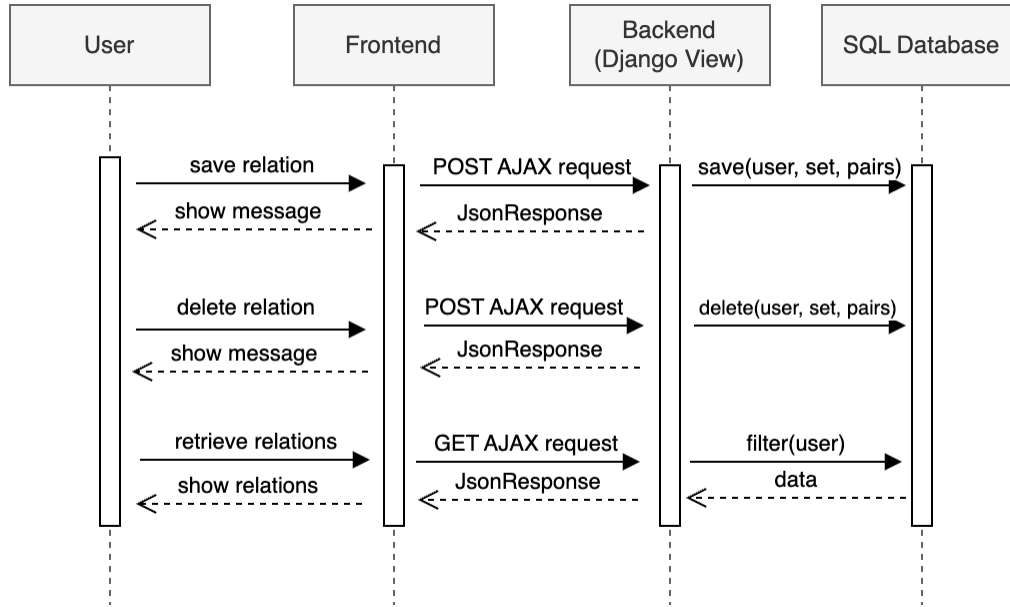


Figure 6.2: Sequence Diagram for Managing Relations via AJAX Requests

As seen in the diagram above, when a user initiates a save or a delete request, an AJAX call is triggered and sends a POST request³ to the Django view that is responsible to process the request. This view communicates with the database and specifies the relation to be saved or deleted by passing as parameters the relation's set, pairs and the user who have created the relation. The database then proceeds to perform the requested operation initiated from the view. Following that, the view sends to the frontend a JsonResponse to indicate whether the operation was successful or not. This outcome is conveyed to the user in the form of success or error messages showing on their screen.

³POST request: HTTP request that sends data to a server.

Similarly, when the user opens any of the three tools, the system automatically retrieves all the relations saved by the user through an AJAX call with a GET request⁴. The associated view is then invoked to retrieve from the database all the relations previously saved by the user. This is achieved by using the Django built-in filter function which we used to filter relations based on the currently logged in user. Once we retrieve the relations associated with the user, we display them in the frontend by listing them in a drop-down menu.

Hence, it can be seen that by using AJAX requests, we allow saving, deleting and retrieving relations from our SQL database without any additional overheads.

6.5 Parsing Relations

To facilitate dealing with relations and their sets programmatically, we have implemented simple parsing functions which are “parseSet(input)” and “parseRelation(r_pairs, r_set)”. When the system receives the set of the relation inputted by the user in a string format, the function “parseSet(input)” validate the set to exclude all invalid symbols or white-space characters. Moreover, as the relation and its set are stored in the database in a string format, the function “parseRelation(r_pairs, r_set)” takes the set and relation as parameters and convert the relation to an array data structure while the set of the relation to a set data structure. Converting the relation to an array data structure allows for direct access and modification to individual pairs in the relation without the need to search the entire string. In addition, converting the set from a string to a set data structure ensures the set is unique without any duplicate elements.

⁴GET request: HTTP request that retrieves data from a server.

6.6 Relation Visualiser Tool

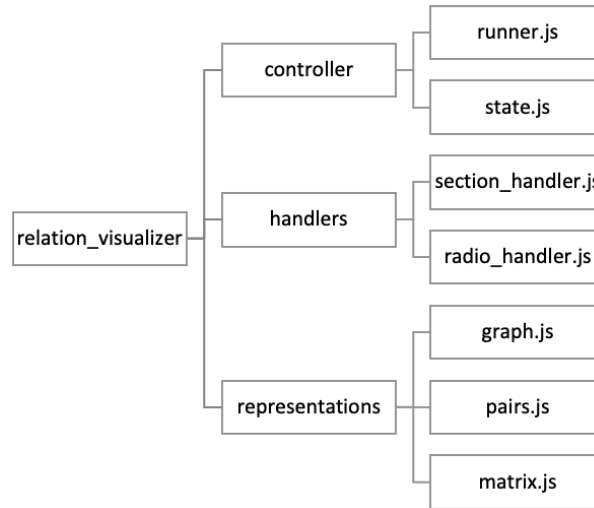


Figure 6.3: Directory Structure for the Relation Visualiser Tool

6.6.1 Relation Representations

As we described in the Requirements and Specification chapter, our system allow users to model their relation with any representation while being able to see the reflected changes in real time on the alternative relation representations. To achieve this, we first structured the tool's page into different sections based on the user representation choice.

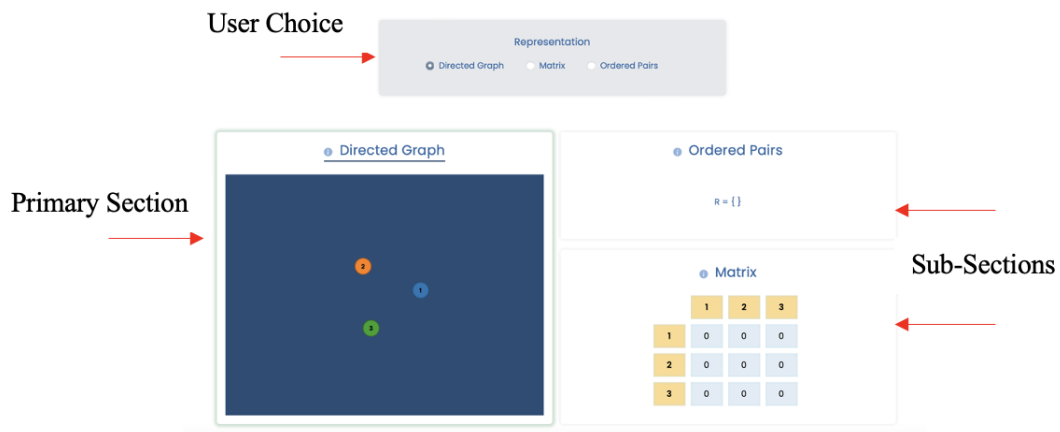


Figure 6.4: Sections Placement Based on User Choice

There are two important handlers that can track the user choice and structure the page accordingly which are: “radio_handler.js” and “section_handler.js”. The “radio_handler.js” contains functions that track the radio button currently checked and based on that current choice, the “placeSection(id)” function in the “section_handler.js” is called to place the representation chosen by the user in the primary section while the alternative representations in the other subsections. Depending on which representation is chosen, the system then either generates a directed graph, matrix or ordered pairs and place it in the primary section. As we can see from figure 6.3, we have the “representations” package which includes the “graph.js”, “matrix.js” and “pairs.js”. Each of these files is responsible for the operations related to a particular relation representation.

Directed Graph

When the user chooses the directed graph representation, the method “executeGraph()” is triggered. This method creates the graph and assigns the nodes with labels based on the elements in the relation’s set. The source code in [33] has been utilised as the building block for creating and setting up the graph. However, the code achieves fairly simple and minimal functionality which only allows connecting nodes with edges. Therefore, in order to satisfy the requirements of our project, we have added further extensions to this source code. These extensions include limiting the number of the generated nodes to the size of the relation’s set, allowing the removal of edges in a particular direction, and adding bolded borders to indicate reflexive pairs when the user double clicks on a node. In addition, the two functions “updateMatrixFromGraph()” and “updatePairsFromGraph()” have been added to update the ordered pairs and matrix representations based on the current state of the directed graph. Lastly, the code has been extended to allow independent drawing of the graph when the user loads a previously saved relation. This has been achieved through the “loadGraph(parsed_r)” function which draws the nodes and their links based on the relation selected by the user.

Matrix

When the user chooses the matrix representation, the function “executeMatrix()” is triggered. This function calls the “createMatrix()” which constructs a table with row and column headers corresponding to the elements in the relation’s set. An event listener is then added to each cell in the table, allowing users to click and toggle the value of individual cells to indicate the presence or absence of a pair in the relation. When a cell is clicked, the “updatePairsFromMatrix()” and “updateGraphFromMatrix()” are automatically triggered to update the state of the ordered pairs and the directed graph, respectively. When users load a relation in the matrix representation, the function “loadMatrixCells(parsed_r)” is invoked to read the user’s selected relation and fills the values of the table cells based on the pairs that are present in the relation.

Ordered Pairs

When the user chooses the ordered pairs representation, the function “executePairs()” is invoked. This function triggers the “createCheckboxes()” function which computes all possible pair combinations that can be formed from the relation’s set, and attach a checkbox to each of those pairs. An event listener is added to each checkbox such that when a user checks or unchecks a box, the methods “updateGraphFromPairs()” and “updateMatrixFromPairs()” are automatically invoked to reflect the changes that occurred to the relation in the graph and matrix representations respectively. In a similar manner to the previous representations, when users select a relation to be loaded in the ordered pairs representation, the function “loadPairs(parsed_r)” is invoked to automatically check the boxes that correspond to the pairs within the user selected relation.

6.7 Warshall Simulator Tool

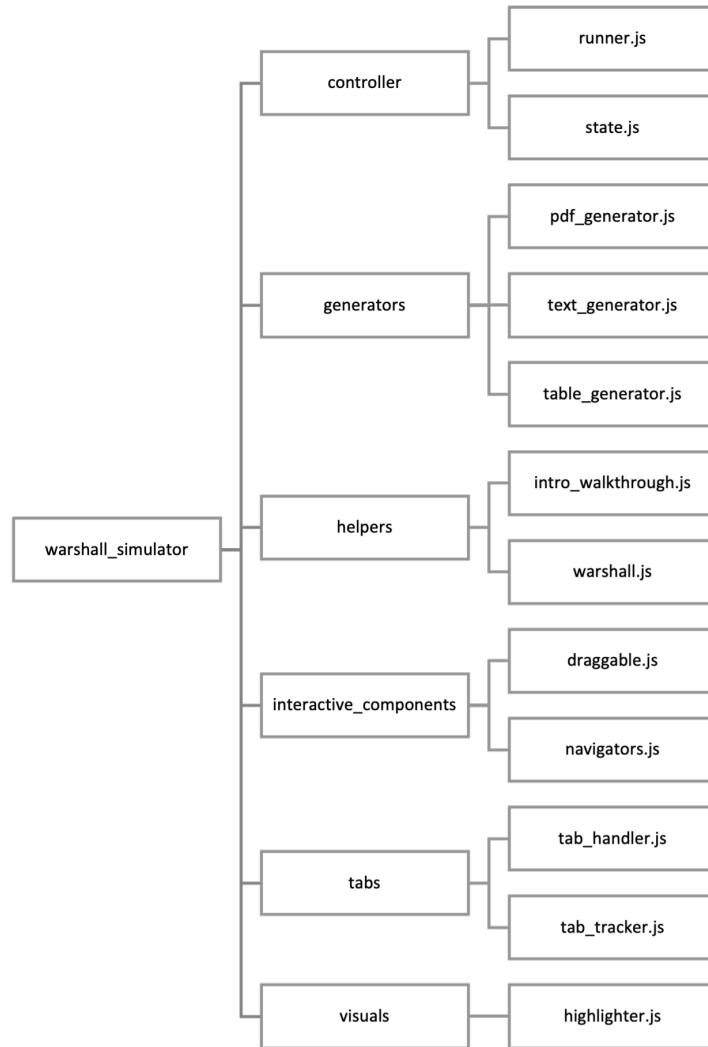


Figure 6.5: Directory Structure for the Warshall Simulator Tool

6.7.1 Algorithm Execution

When the user creates a new relation or loads a previously saved relation and attempts to execute the Warshall algorithm, the system initiates a set of sequential operations. Firstly, the function “readRelationFromMatrix()” is invoked to extract the relation from the matrix cells. The extracted relation is stored as a binary representation in a two-dimensional array. Secondly, the function “warshall(arr)” runs the Warshall algorithm on the two-dimensional array generated from the prior step and stores the results of each round in a dictionary

called “output”. This dictionary data structure allowed us to store key-value pairs using the round number as the key and the resulting matrix for that round as the corresponding value. Additionally, the function closely monitors the cells that have changed from 0 to 1 in each round. This has been achieved by storing the coordinates of these cells in a dictionary called “change_detector”. The dictionary stores the round number as the key and the coordinates of the cells that have changed in that round as the corresponding value. Lastly, we invoke the function “generateExplanation()” which uses our “change_detector” dictionary to generate an explanation text for each round of the algorithm. This explanation specifies which cells have been changed within each round along with a justification for why the change has occurred.

We should note that our implementation of the “warshall(arr)” function runs the algorithm once and stores the resulting matrices from each round rather than computing these matrices during the user navigation between the different steps of the algorithm. Taking this approach has resulted in reducing the computational overhead required for the algorithm’s execution and ensured a smoother navigation.

6.7.2 Navigation

To enable users navigate between the different rounds of the algorithm, we created a variable called “round” to help us track the round of the algorithm we are currently at. By tracking the round number, we were able to display the appropriate matrix on the screen. As mentioned earlier, the resulting matrices from each round are pre-computed and stored in the “output” dictionary. This means, when the user clicks on the “next” button, the “next()” function is invoked to increment the “round” value and retrieves the matrix of the following round from our “output” dictionary. Similarly, when the user clicks on the “previous” button, the “previous()” function is invoked which decrements the “round” value and gets the matrix of the preceding round. Additionally, to provide a further level of interactiveness, we allow users to play the algorithm at a particular speed in fast forward mode. This functionality is achieved through the “playFastForward()” function, which repeatedly calls the “next()” function until the algorithm completes execution, taking into account the user-specified speed by setting time intervals between each of these calls. Users may also halt the fast forward execution at any point by clicking the “pause” button. This action triggers the “pauseFastForward()” function which terminates the execution while preserving the current state of the algorithm.

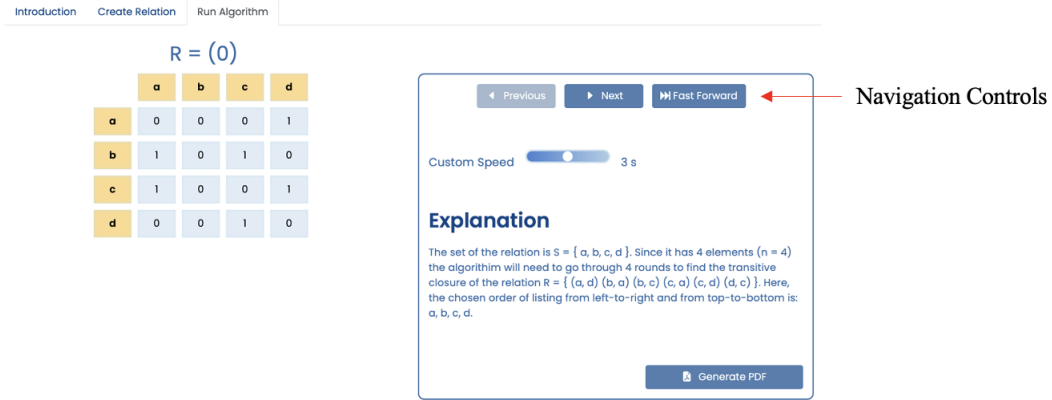


Figure 6.6: Algorithm Navigation Controls

6.7.3 Visualisation

Visualisation has been thoughtfully integrated into this tool. One means of enhancing the user learning experience and aiding in the visualisation of the algorithm is to highlight the relevant row and column at each round of execution. However, implementing this visual feature presented a challenge since there is no built-in way to select or style an entire column in a regular HTML table. To address this challenge, we have devised a solution by converting our standard HTML table into a custom table constructed using only HTML `<div>` tags and CSS properties. This conversion was achieved through the "table2div(table, parent)" function, which creates a custom table that replicated the appearance of the standard table. By adopting this solution, we have gained greater control and flexibility over the table elements, allowing us to highlight rows and columns in a visually appealing way.

Introduction Create Relation Run Algorithm

$R = (1)$

	a	b	c	d
a	0	0	0	1
b	1	0	1	1
c	1	0	0	1
d	0	0	1	0

Figure 6.7: Row and Column Highlighting for Improved Visualisation

Another issue related to the highlighting of rows and columns arises when the size of the table increases, making it difficult to track which column is being highlighted at a particular round of execution. To solve this issue, the function “focusOnColumn()” has been implemented. This function automatically scrolls the table to the column currently being highlighted, allowing the user to easily see it without having to manually scroll through the table. With this addition, we minimise the risk of users losing track of the algorithm’s progress and ensure that they can easily follow the execution of the algorithm when working with large relations.

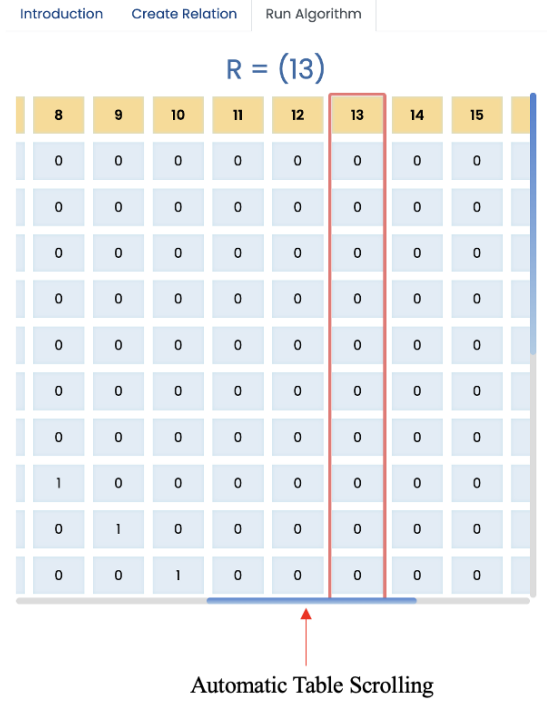


Figure 6.8: Automatic Table Scrolling to a Highlighted Column

Another area where visualisation was incorporated is by adding a flashing effect to the cells that have changed from a value of 0 to 1. As previously mentioned, our “warshall(arr)” function tracks and stores the coordinates of cells which their value has changed in the subsequent round using the “change_detector” dictionary. Thus, to add this visual feature, we implemented the function “highlightCells()”, which utilises this dictionary to locate the changing cells based on the current round and adds a flashing effect to it. Moreover, we implemented the “addToolTips()” function which displays a tool-tip with a text explaining why the cell’s value has changed when the user hovers over it.

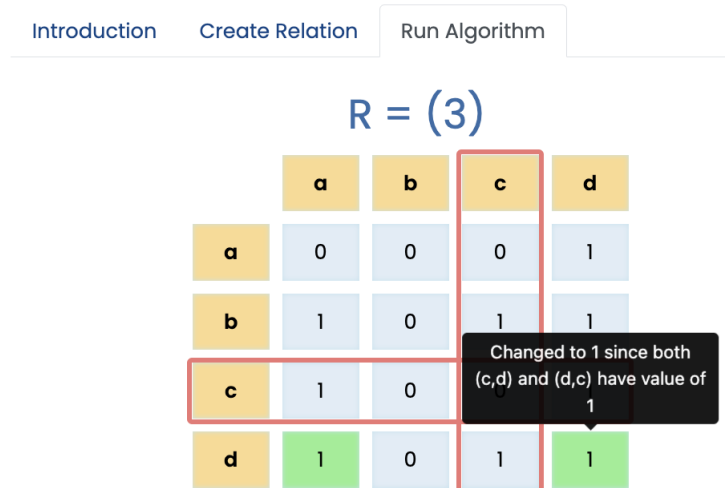


Figure 6.9: Flashing Cells with Feedback Tool-tips on Hover

6.7.4 PDF Generator

In order to add a notable convenience to the users' learning experience, the system offers the option to generate a PDF document containing a summary of the algorithm's execution. To implement this feature, we created the "pdf-generator.js" file which contains the essential functions for generating and formatting the PDF document. Additionally, we created the "text-generator.js" file which includes functions for generating nicely formatted explanations on each round of the algorithm's execution. When the user initiate a generate request, the "generatePDF()" function is invoked to create the document. It iterates over every round of the algorithm and constructs the matrix (i.e. a table) resulting from each round. The function also adds a descriptive text to the document to explain the changes that has occurred within a particular round by invoking the relevant functions from our text generator file.

Furthermore, to enhance the visual appeal of the generated document, we applied colour-coding to the cells that have changed in each round. We utilised our "change_detector" dictionary to locate and identify these cells.

0	0	0	1
1	0	1	1
1	0	0	1
1	0	1	1

Figure 6.10: Colour-coding on Changing Cells

However, as with many well-known text editing environments, there is a limit on the number of rows and columns to generate a table. In our project, we implemented the function “checkPDFLimit()” to adopt a similar limit check prior to generating the PDF document. The function checks the size of the set and only generates the document if the set contains no more than 20 elements. This check is necessary to ensure that the size of the generated table is appropriate for fitting into a standard A4 page.

◀ Previous

▶ Next

⏭ Fast Forward

Custom Speed

3 s

Explanation

The set of the relation is $S = \{ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21 \}$. Since it has 21 elements ($n = 21$) the algorithm will need to go through 21 rounds to find the transitive closure of the relation $R = \{ (1, 12) \}$. Here, the chosen order of listing from left-to-right and from top-to-bottom is: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21.

📄

Generate PDF

Limit to generate PDF is exceeded

→

* To generate PDF file the set must contain at most 20 elements.

Figure 6.11: Limit Check for the PDF Generation Feature

55

6.8 Relation Explorer Tool

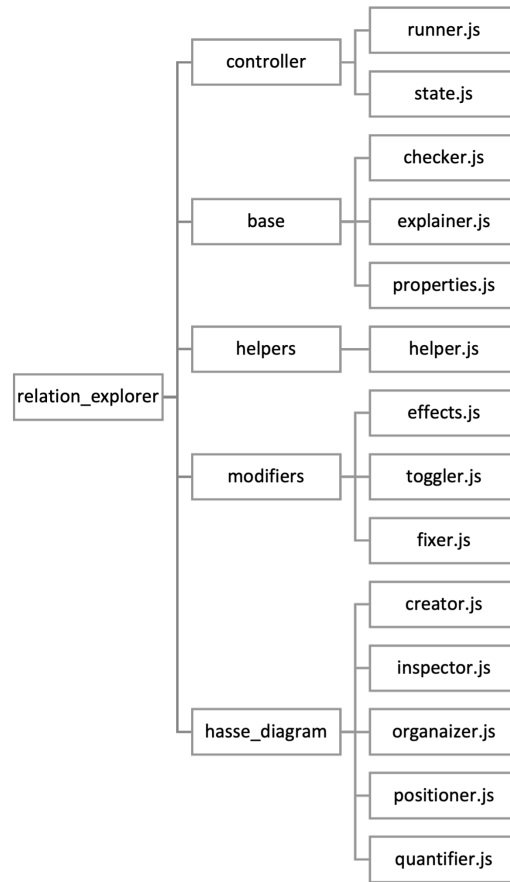


Figure 6.12: Directory Structure for the Relation Explorer Tool

6.8.1 Properties Identification

When users create or load a relation, our system identifies the properties that are satisfied by that relation. This is achieved by implementing a number of essential functions within the “properties.js” file. These functions include “isReflexive(A, R)”, “isSymmetric(A, R)”, “isAntiSymmetric(A, R)”, and “isTransitive(A, R)”. The functions take the set and relation as inputs and check for the reflexivity, symmetry, anti-symmetry, and transitivity properties respectively. The results returned from the functions are maintained in a dictionary called “properties” which stores the property identifier as the key and the function result as the value. By storing these results, we can determine whether the properties holds on a relation in different parts of the code without the need to rerun the aforementioned functions.

6.8.2 Properties Enforcement

To help enhance the user’s learning experience and improve the overall flexibility of the website, we have implemented a feature that enables users to enforce unsatisfied properties. To achieve this, we created the “fixer.js” file which contains the functions “fixReflexivity()”, “fixSymmetry()”, “fixAntiSymmetry()”, and “fixTransitivity()”. These functions fix the relation to satisfy the reflexivity, symmetry, antisymmetry and transitivity properties, respectively. When a property is not satisfied, we update the graphical user interface with a cross-mark next to the unsatisfied property. With the help of some CSS attributes, we also distinguish the box associated with the unsatisfied property with a shadow effect to indicate that it is clickable. When clicked, the corresponding fixer function will be invoked to fix and update the relation.

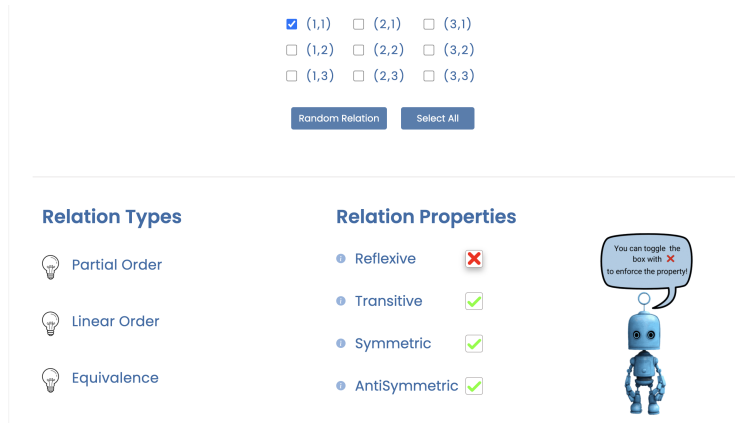


Figure 6.13: Before Enforcing the Reflexivity Property

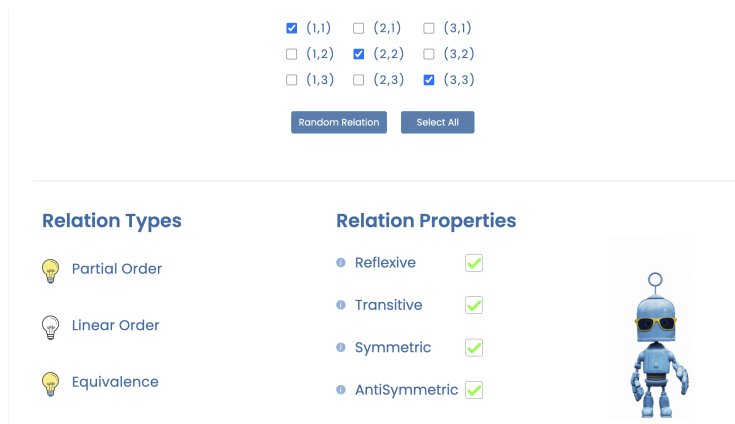


Figure 6.14: After Enforcing the Reflexivity Property

Moreover, we store and maintain counterexamples that led a property to being unsatisfied in a dictionary called “counter_examples”. Typically, the fixer functions we mentioned above add or remove pairs from the relation based on the counterexamples we stored in this dictionary. However, to enforce the transitivity property, we require an extra check to ensure the relation remains transitive while it is being fixed. This check requires iterating until the relation satisfies the transitivity property. The code snippet below shows the “fixTransitivity()” function which repeatedly updates the relation with the missing transitive pairs until it ultimately becomes transitive and the loop terminates.

```
function fixTransitivity(){
  state.pairs['T'] = [];
  while(!state.properties['T']){
    let pairs = [];
    for (let i = 0; i < state.counter_examples['T'].length; i++) {
      /* The counter examples for transitivity are represented in
         the form of a list of three pairs i.e. [[A,B], [B,C], [A,C]],
         the third pair, represented by index 2, is the missing transitive pair.
      */
      pairs.push(state.counter_examples['T'][i][2])

      // some lines are omitted
    }
    modifyCheckBoxesByValues(pairs, true);
    updateRelation();
    showLabels();
  }
}
```

Figure 6.15: Code Snippet for the fixTransitivity() Function

To illustrate the need for this iteration, suppose the user create the relation $R = \{(1,2), (2,2), (2,3), (3,3), (4,1), (4,4)\}$. Based on the current state of the relation, the counterexamples associated with the transitivity property will only consist of the pairs (1,3) and (4,2). To see why these pairs violate the transitivity property, notice that our relation contains the pairs (1,2) and (2,3) but not (1,3). Similarly, the relation contains the pairs (4,1) and (1,2) but not (4,2). However, by adding only these two pairs, we find that the transitivity property is still violated. This is because by the time we add (4,2), it becomes necessary to also add (4,3) as both (4,2) and (2,3) are now in the relation. It is only after the addition of (4,3) the transitivity property will hold and our loop terminates.

Overall, adding this properties enforcement feature helps us offer a user-centred learning that prioritises flexibility and convenience.

6.8.3 Property Status Update

To ensure that our system updates the state of the properties in a timely manner as users modify their relation, we attached an event listener to each checkbox. This listener triggers the functions we mentioned in section 6.8.1 to check which properties are satisfied and updates their status in the graphical user interface whenever a pair is added or removed from the relation. Additionally, to assist and guide users in their learning, we added tool-tips to explain why the properties hold or not. The “explainer.js” file contains the functions that generate the text associated with these tool-tips. In case of an unsatisfied property, we update the tool-tip text to include a counterexample to justify the reason to why a property is not satisfied. This text is displayed when the user hover over the information icon next to each property.

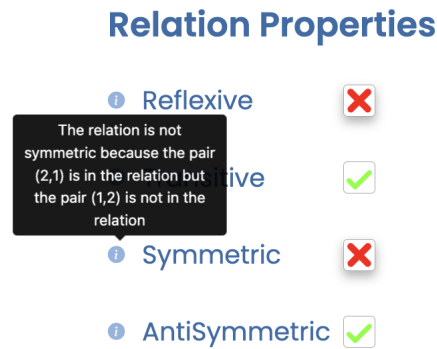


Figure 6.16: Tool-tip: Counterexample for the Symmetry Property

6.8.4 Classification of Relations

To further augment the educational value of our tool, we have classified relations into an equivalence, partial, and linear order. This has been achieved by implementing three functions: “isEquivalence(A,R)”, “isPartialOrder(A,R)” and “isLinearOrder(A, R)”. These functions take the set and relation as inputs and access the “properties” dictionary to determine which properties are currently satisfied and classify the relation accordingly. However, it should be noted that the implementation of the “isLinearOrder(A, R)” function is quite different since it does not only check if the required properties for a linear order are satisfied, but also verifies if the linearity condition is met.

6.8.5 Hasse Graph

To construct the Hasse graph of the relation, we created multiple interconnected files. Each of these files contains functions that communicate together to achieve the overall goal of constructing the Hasse graph. For instance, the “creator.js” file creates and sets up the graph by adding the nodes and edges. The “organizer.js” file contains functions that shift, add and assign the nodes to different levels. The “inspector.js” file contains some necessary functions for identifying the source and target nodes for a given node that are important to determine which level a node should be assigned to. In addition, the “positioner.js” file is responsible for arranging the nodes within the graph based on their interrelationships with one another. Finally, the “quantifier.js” file contains functions for calculating critical dimensions such as the gaps between the nodes in the x and y directions, ensuring an aesthetically pleasing graph.

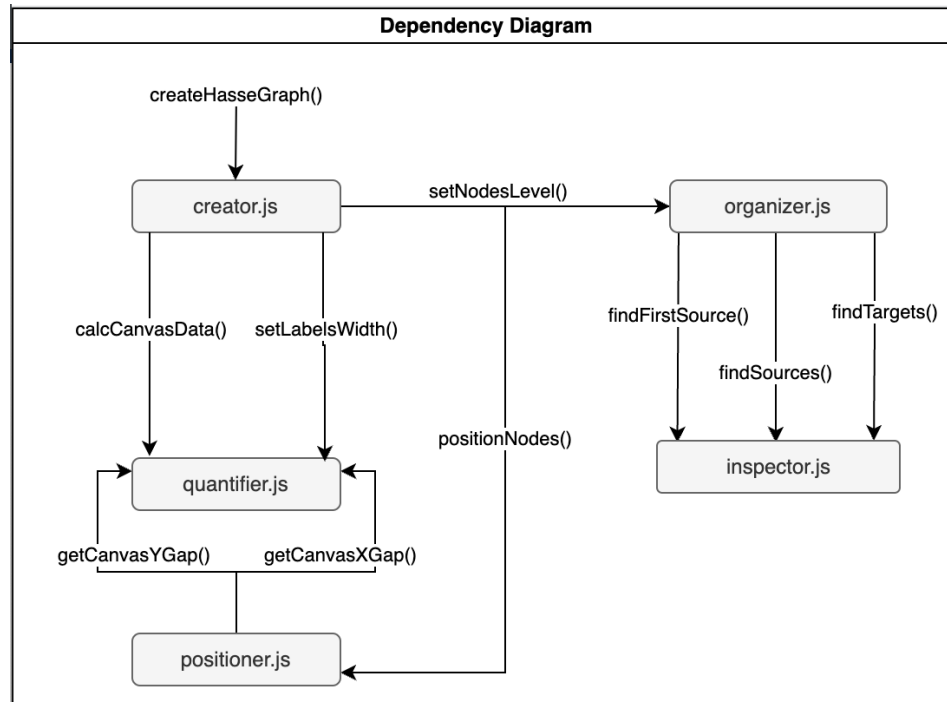


Figure 6.17: Dependency Diagram for the Hasse Graph Construction

The dependency diagram above shows how these files interact with one another and demonstrates the process flow involved in creating the Hasse graph. The process starts with the invocation of the “`createHasseGraph()`” function in the “creator.js” file. This function generates the graph and triggers other functions in the “quantifier.js” file to calculate the dimensions of the graph canvas and determine the width of the nodes’ labels. Once the graph is set up, the function “`setNodesLevel()`” is invoked to set the nodes into different levels within the graph.

This is achieved by utilising the functions contained within our “inspector.js” file which can help us identify the sources and targets of a node; thereby determining the appropriate level for each node. Once the nodes are assigned to different levels, the “positionNodes()” function is invoked to arrange and position the nodes in the graph. This positioning procedure is achieved with the help of the functions contained in the “quantifer.js” file which quantifies and determines the appropriate spacing between the nodes in the x and y directions.

Graph Setup

To set up the required elements of the graph, we have used the Cytoscape.js library to help us draw the nodes and edges. We have then focused our efforts on customising the graph to fit the specific requirements of the project. For example, as discussed in the Background section, a Hasse graph must exclude reflexive and transitive edges. Therefore, in the implementation of the “createHasseGraph()” function, we loop over all the pairs in the relation and examine them cautiously to ensure that the edges we add are not implied by the reflexivity or transitivity properties. These checks occur in the code in line (17) and (22). We also identify the nodes which are only connected to themselves (i.e. do not have any sources or targets) and place them on the first level of the graph as demonstrated in lines (32 - 33).

```

9  function createHasseGraph() {
10  // some lines are omitted
11  let count = 0;
12  for (let i = 0; i < state.parsed_r.length; i++) {
13    let node1 = state.parsed_r[i][0];
14    let node2 = state.parsed_r[i][1];
15
16
17    if (node1 == node2) {
18      reflexive_nodes.push(node1);
19    }
20
21
22    else if (!doesPairExist(state.transitive_pairs, [node1, node2])) {
23      addEdge(node1, node2);
24      state.edges[count] = { source: node1, target: node2 };
25      count++;
26    }
27  }
28
29  reflexive_nodes.forEach((node) => {
30    let sources = findSources(node).length;
31    let targets = findTargets(node).length;
32    if (sources === 0 && targets === 0) {
33      addNodeToLevel(0, node);
34    }
35  });
36  // some lines are omitted
37 }

```

Figure 6.18: Code Snippet of the createHasseGraph() Function

Allocating nodes to levels

The nodes in a Hasse graph should be arranged into different levels such that the target node is located at a higher level than its source node. This process requires us to examine the source and target nodes of a given node to determine which level it should be assigned to. The code to accomplish this task becomes increasingly complicated as it requires taking into account multiple cases.

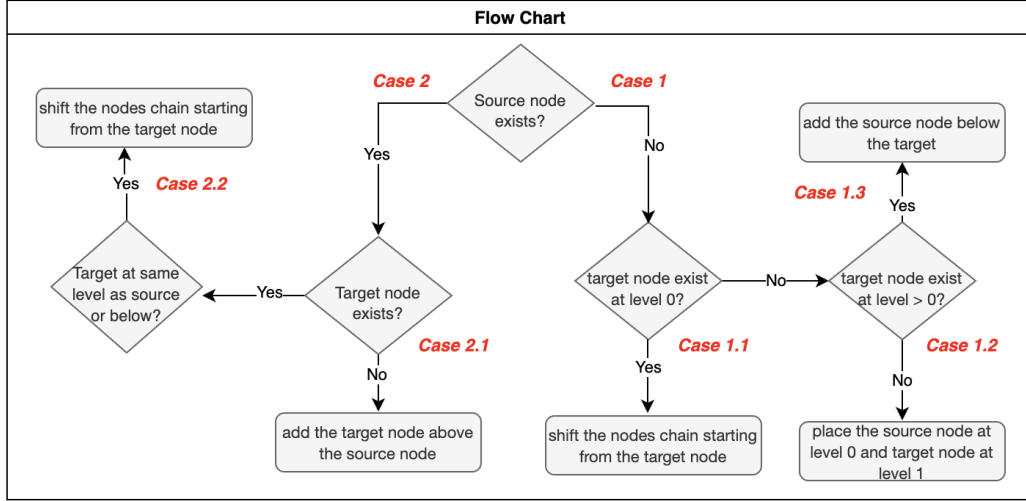


Figure 6.19: Flowchart for Node Level Assignment

The diagram above shows the flow of our “setNodesLevel()” function. We begin by checking if the source node is already present in the graph. If not, we proceed to analyse its target node. If the target is placed at level 0, we move it and all its successors one level upward to place the source node below it. On the other hand, If the target node is at level higher than 0, we simply add the source node on the level below it. If neither the target nor the source node has been added to the graph yet, then we place the source node at level 0 and its target directly above it, which is at level 1. However, If the source node exists, we will have to deal with different cases. In case the target was not yet present then we simply add the target above the source. If the target exist and is situated at the same level or below the source node, we then shift the chain of nodes starting from the target by one level upward. This chain shifting is necessary to allow us place the source node below its target.

To illustrate how the code works, suppose a user creates the relation $R = \{(A, A), (A, B), (A, G), (B, B), (B, G), (C, A), (C, B), (C, C), (C, G), (D, A), (D, B), (D, D), (D, G), (G, G)\}$ using the set $\{A, B, C, D, G\}$. Before building the graph, we begin by identifying and removing the reflexive and transitive edges, resulting in the new relation $R = \{(A, B), (C, A), (D, A), (B, G)\}$. Now, when we add the edge between A and B, we will enter Case 1.2 as indicated in figure 6.19 since initially both the target and source nodes do not exist yet. The resulting graph will be as shown below.

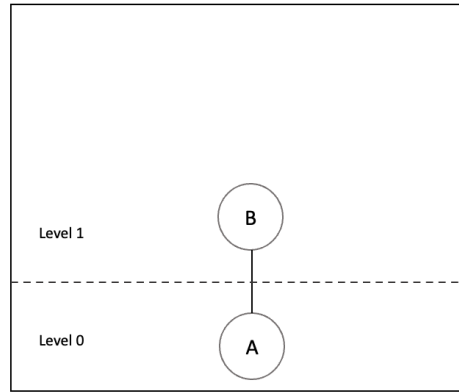


Figure 6.20: Example of Case 1.2

When adding the edge between C and A, we will enter Case 1.1 since the source node does not exist and its target is currently at level 0. The resulting graph will be as shown below.

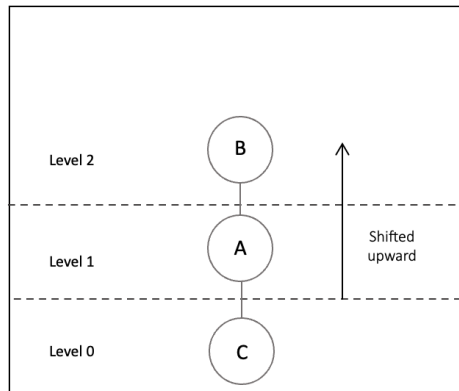


Figure 6.21: Example of Case 1.1

When adding the edge between D and A, we will enter Case 1.3, since the source node does not yet exist while the target node exists at level greater than 0. The resulting graph will be as shown below.

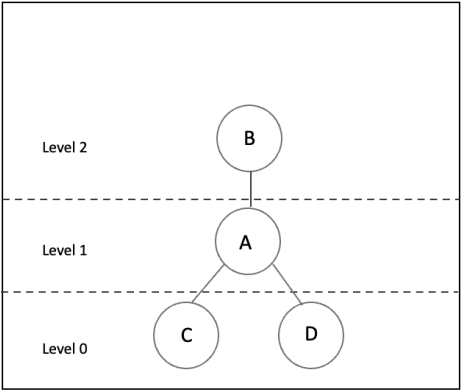


Figure 6.22: Example of Case 1.3

When adding the edge between B and G, we will enter the Case 2.1 since the source node exists while the target does not yet exist. The resulting graph will be as shown below.

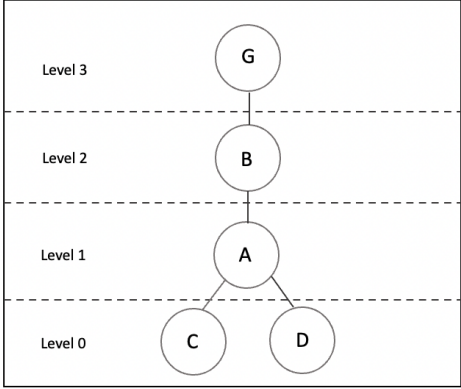


Figure 6.23: Example of Case 2.1

Finally, to depict Case 2.2, where the source and target nodes are both situated at the same level, suppose the user decides to add the pair (C,D) to their relation. It can be noticed from the previous figure, both nodes are located at level 0. This means we will have to shift the chain of nodes starting from D in order to place node C below it. However, after adding this pair, we will need to remove the transitive edge (C,A) which will result into the updated relation $R = \{(A,B), (D,A), (B,G), (C,D)\}$. Hence, the final resulting Hasse graph is shown below.

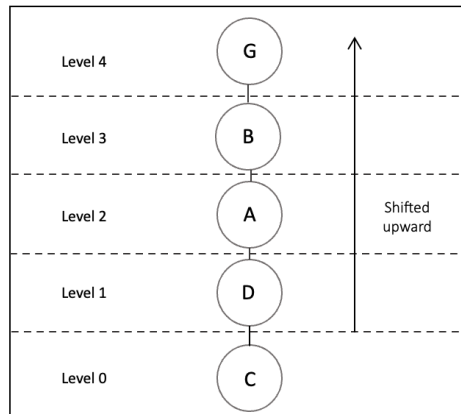


Figure 6.24: Example of Case 2.2

Adjusting Nodes Within Level

To improve the clarity of the graph, the implementation has been extended to minimise the occurrences of intersecting edges. While the Hasse graph allows edges to intersect, avoiding them, when possible, was deemed important to allow for a better comprehension and visualisation to the learner. The logical solution that has been taken to address this issue is implemented in the “adjustNodesWithinLevel()” function. The general idea of the solution is to look for the nodes that can potentially cause intersecting edges, store their indices, and perform swapping to eliminate such intersections.

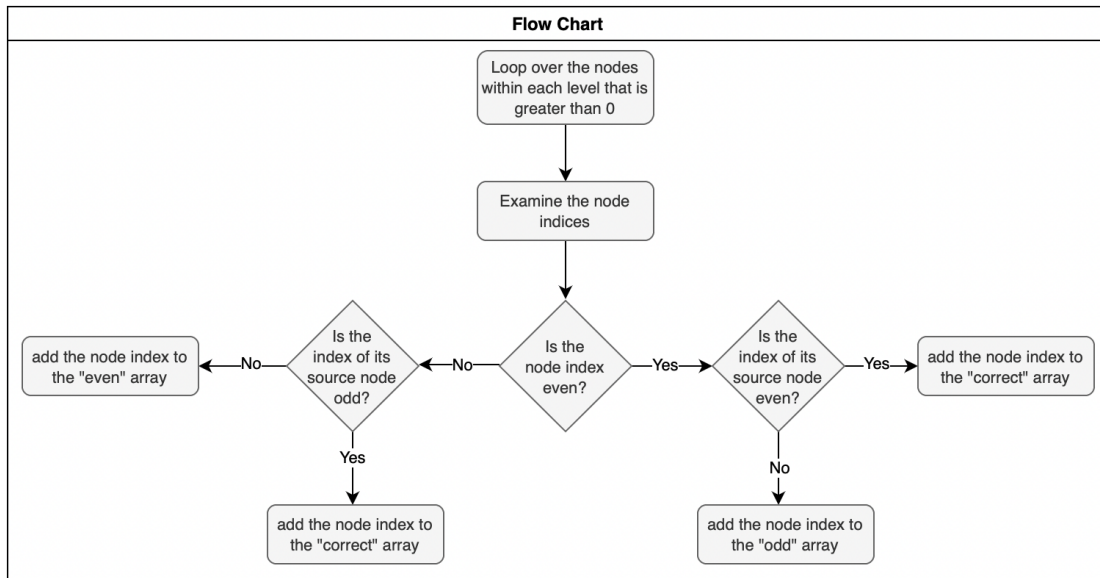


Figure 6.25: Flowchart of Node Index Assignment

As demonstrated in the flowchart above, we begin by looping over the nodes within each level higher than 0 and start filling three arrays: “correct”, “even” and “odd”. The “correct” array stores the node indices that are placed in the correct position within a level. A node index is added to the “correct” array when the node index and its source node index are both even or both odd. The “odd” array stores the node indices that are even but their source node is placed at an odd index while the “even” array stores the node indices that are odd but their source node is placed at an even index.

Once our loop terminates, we look for potential swappings that we can perform to eliminate intersecting edges. The idea is that we check the index of the node, if it exists in the “correct” array, this means our node is positioned correctly and does not need to be swapped. If the node index is not stored in the “correct” array, then we will need to check if a swapping is possible depending if there are entries within the “odd” and “even” arrays that we can swap our node with. The following flowchart demonstrates our swapping logic.

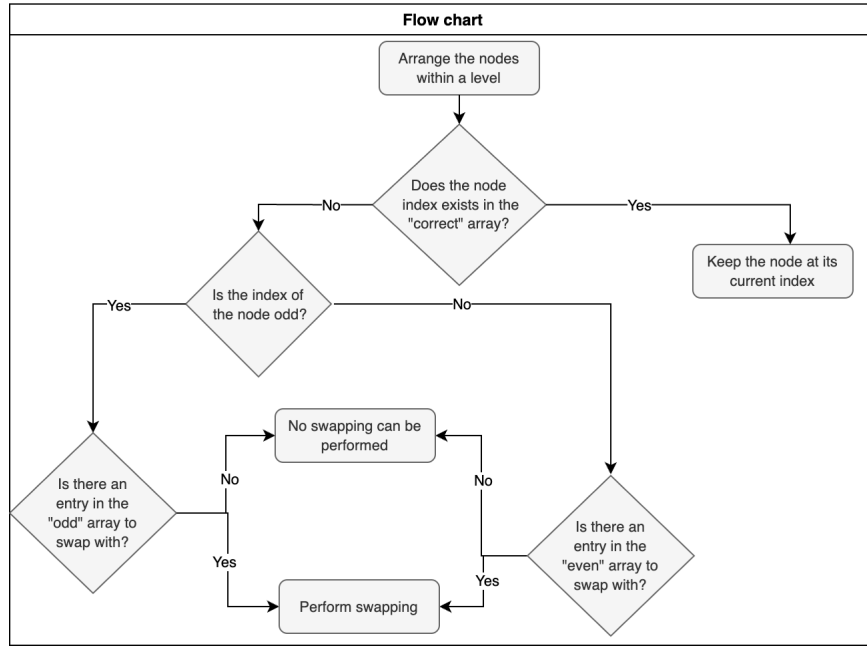
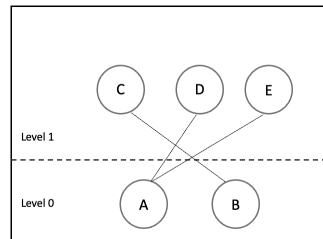
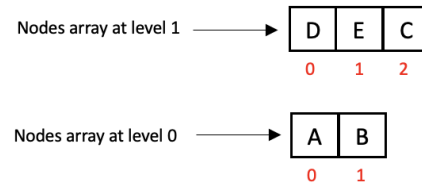


Figure 6.26: Flowchart of Node Swapping

To simplify the logical sequence depicted in the flowcharts shown in figure 6.25 and 6.26, we will demonstrate the procedure with an example. Given the initial configuration of the graph shown in figure 6.27 (a), we will execute the function “adjustNodesWithinLevel()”. The function begins by looping over the nodes within each level that is greater than 0, which in our case, will only be level 1. We start by examining node D. Since the node index is 0 (even) and its source node A, is at index 0 (even), we will insert node D’s index in the “correct” array. Next, we will examine node E. The node is at index 1 (odd) and its source node A, is at an even index, then we will insert the index of node E in the “even” array. Finally, we will analyse node C. Its index is 2 (even) and its source node B, is at index 1 (odd), then we will insert the index of node C in the “odd” array.



(a) Initial Graph



(b) The Indices of Nodes in Each Level

Figure 6.27

The content of our three arrays when the loop terminate will be:

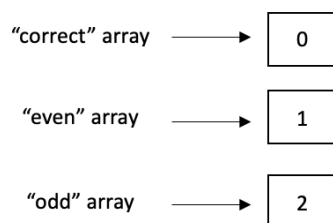


Figure 6.28: Arrays Content After the Node Index Assignment Process

After populating the three arrays with values, we examine if we can do any potential swapping between the nodes. Starting from node D, we can observe that its index is in the “correct” array. Hence, we will keep it at its current position. We then examine nodes E and C. We find that their indices are stored in the “even” and “odd” arrays respectively. Therefore, we will swap their positions. The swapping will result in a new arrangement of the nodes, which eliminated the intersecting edges we had at the start.

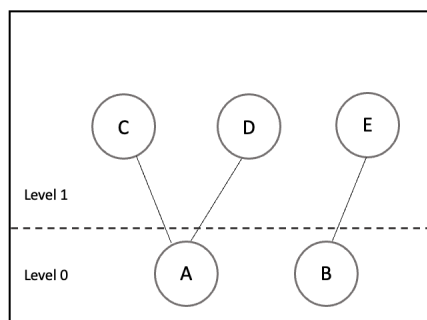


Figure 6.29: Result of Node Swapping: Hasse Graph with Non-Intersecting Edges

Node Positioning

To ensure a visually appealing look of the graph, extra measures have been taken when positioning the nodes. This has been achieved with the “positionNodes()” function which first determines where a node should be placed (i.e. right or left) based on its index. It then computes the optimal horizontal and vertical gaps between the nodes using the functions “getCanvasX-Gap(level)” and “getCanvasYGap(level)”. Based on these computed values, the final positions of the nodes are set by invoking the function “setNodePosition(node,x,y)” which places the node at the specified x and y coordinates in our canvas. The implementation of this function helped us produce a clear and readable graph by avoiding overlapping between the nodes.

Graph Type Switcher

A simple yet effective extension to the tool is to provide users with the option to switch between the Hasse graph and the original directed graph of the relation. To achieve this, we created the variable “graph_state” which help us identify the type of graph currently showing. This variable stores two values which could be either “hasse” or “directed”. When the Hasse graph is currently displayed and the user switches to the directed graph, the function “switchOriginalGraph()” is invoked. This function removes the Hasse graph from the canvas and draw the directed graph by invoking the function “createDirectedGraph()”. The function also updates the value stored in the “graph_state” variable to “directed” to indicate that the directed graph is currently being displayed. Similarly, when the user switches to the Hasse graph, we invoke the function “switchHasseGraph()” which removes the directed graph from the canvas, updates the value of the “graph_state” vairable to “hasse”, and calls the “createHasseGraph()” function to draw the Hasse graph again.

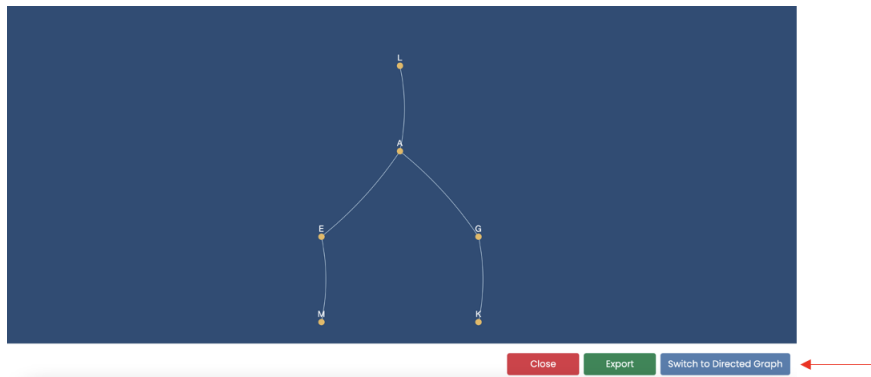


Figure 6.30: Hasse Graph

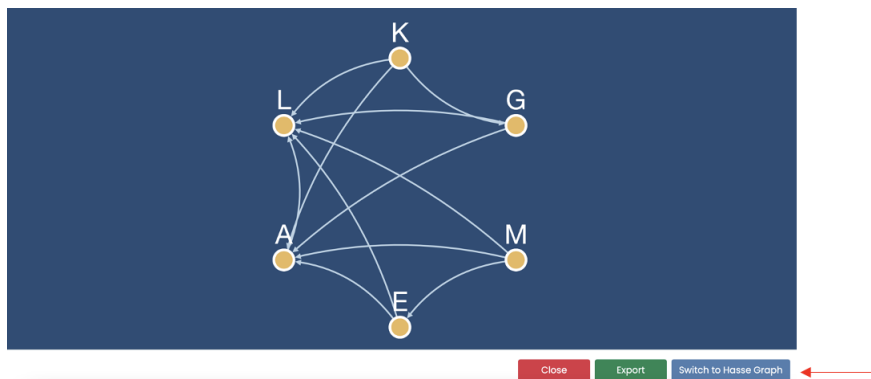


Figure 6.31: Directed Graph

Chapter 7

Testing

7.1 Brief

Testing is a crucial stage in software development. It ensures the system satisfies its requirements and functions as intended. In this project, we performed various forms of testing to help us validate the functionalities offered by the system. This chapter will begin by describing the automated unit testing that has been done to test the Python and JavaScript code, highlighting the tools and frameworks used. It will then introduce the manual testing that has been carried out, including the test methodology, cases and outcomes. The chapter will then conclude with our integrated test suite. By showing our testing results, we aim to demonstrate the quality of the software developed and verify that we have successfully met the project requirements.

7.2 Automated Unit Testing

7.2.1 Python Code Testing

To test the python components of the project, the Django testing framework has been used. With the help of this framework together with the unittest library, we were able to test our Django components. These components includes the views that are responsible for processing the data and rendering our HTML pages, the forms, which are used to allow users sign up, login or reset their password, and the models which we use to store the data of our users and their relations. To test and validate these three components, we have conducted a total of 91 automated tests. Furthermore, to evaluate the coverage of our tests, we used the coverage library to help us identify parts of the code that were not covered in our test cases.

```
DiscreteMathTutor — -zsh — 155x38
[(venv) DiscreteMathTutor % python3 manage.py test
Found 91 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 91 tests in 23.485s

OK
Destroying test database for alias 'default'...
```

Figure 7.1: Result After Running the Python Tests

Coverage report: 100%

coverage.py v7.2.1, created at 2023-03-04 21:02 +0300

Module	statements	missing	excluded	branches	partial	coverage
core\forms.py	53	0	0	10	0	100%
core\mixins.py	13	0	0	2	0	100%
core\models.py	14	0	0	0	0	100%
core\views.py	114	0	0	12	0	100%
Total	194	0	0	24	0	100%

coverage.py v7.2.1, created at 2023-03-04 21:02 +0300

Figure 7.2: Results of the Test Coverage Report

7.2.2 JavaScript Code Testing

To test our frontend and backend logic which is developed using JavaScript, we utilised the Qunit Testing framework. This framework allowed us to create automated unit tests that can run in a browser environment. The tests were separated into four main test suites. The first test suite includes tests that cover the application main features such as parsing, and input validation. It also consists of testing the functionalities related to the frontend components such as the dropdown menu, buttons and feedback messages. The second test suite concentrated on testing the Relation Visualiser tool while the third test suite focused on testing the Warshall Simulator tool. Finally, the fourth test suite was dedicated to test the Relation Explorer tool. The test cases have been carefully written to capture various scenarios where the user can interact with the site. Moreover, to further organise the tests, different modules have been created inside each of our four test suites. These modules facilitate access and navigation through the tests by grouping related test cases together.

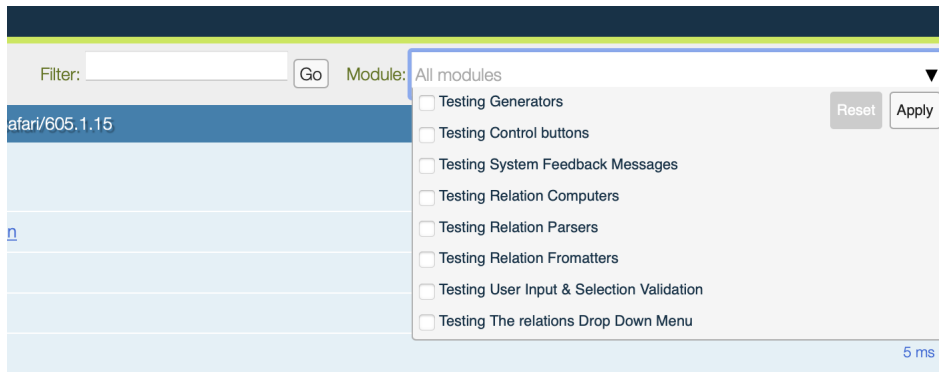


Figure 7.3: Division of Tests into Modules within the Test Suite

The four figures below show a partial view of the test reports with a highlighting on the number of tests conducted, the number of assertions made, and the test results obtained.

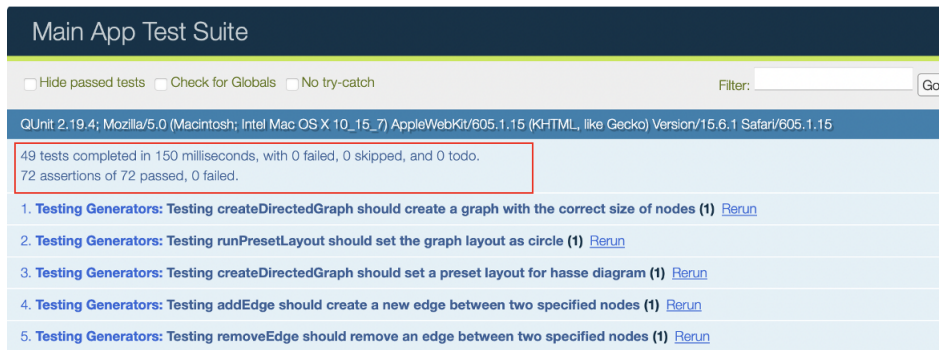


Figure 7.4: Test Suite for the Application Main Features

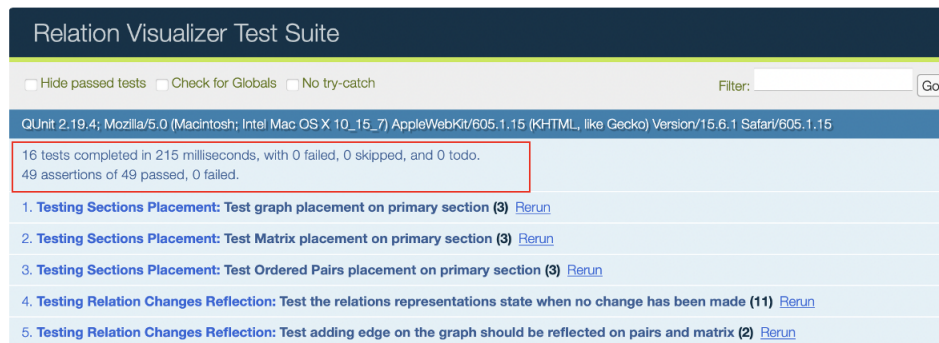


Figure 7.5: Test Suite for the Relation Visualiser Tool

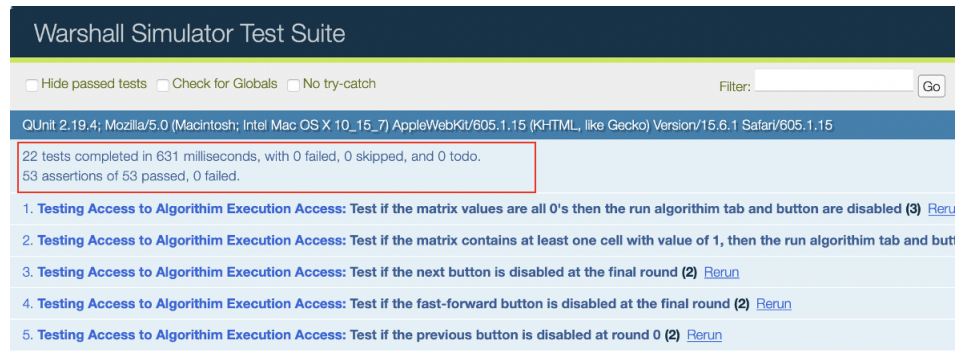


Figure 7.6: Test Suite for the Warshall Simulator Tool



Figure 7.7: Test Suite for the Relation Explorer Tool

7.3 Manual Testing

In addition to automated testing, extensive manual testing has also been carried out.

The testing methodology used is black box, which involved examining the system under different inputs and scenarios to identify issues that could arise from the perspective of the user. The tests were structured similarly to the automated tests, dividing them into four main components to cover the application main features and each of our three learning tools. The results of these tests have been documented in a table structure which included the test case name, an explanation of the test case, the test data used as input and the resulting test status which can be either pass or fail. For a detailed description of the manual test cases, please refer to appendix (A.2).

7.4 Integrated Test Suite

To allow easy access and navigation to our coverage and test reports, an integrated test suite page has been created to gather all the tests into a single and centralised location. This page is not part of the website because regular users cannot access it. However, admin users are granted access and navigation through this page. To be able to access the test suites please refer to appendix (B.1.2) for further information.



Figure 7.8: Integrated Test Suite Page

Overall, with our combined approach of automated and manual testing, we ensured a thorough and comprehensive evaluation of the system's requirements.

Chapter 8

Evaluation

8.1 Brief

This chapter will provide a thorough evaluation of the system's performance, accessibility and compatibility. Following the evaluations, the chapter will conclude with a critical analysis of the system, highlighting its key strengths and potential areas of improvement.

8.2 Performance Evaluation

Performance is a critical evaluation measure in this project. This is because our project revolves around the idea of providing an interactive and responsive learning website. A website that is not performing optimally or suffers from major delays when loading a page or responding to user requests undermines our interactivity objective. Therefore, evaluating the website's performance was a necessary step to take. To achieve this, we have utilised Google Lighthouse¹, an open-source tool that can audit and measure the accessibility, performance, and best practices of a web-page. Lighthouse performance score is based on six metrics. These metrics include First Contentful Paint (FCP), Time to Interactive (TTI), Speed Index (SI), Total Blocking Time (TBT), Largest Contentful Paint (LCP), and Cumulative Layout Shift (CLS).

¹<https://developer.chrome.com/docs/lighthouse/overview/>

Lighthouse Scoring Calculator

Device type: Versions:

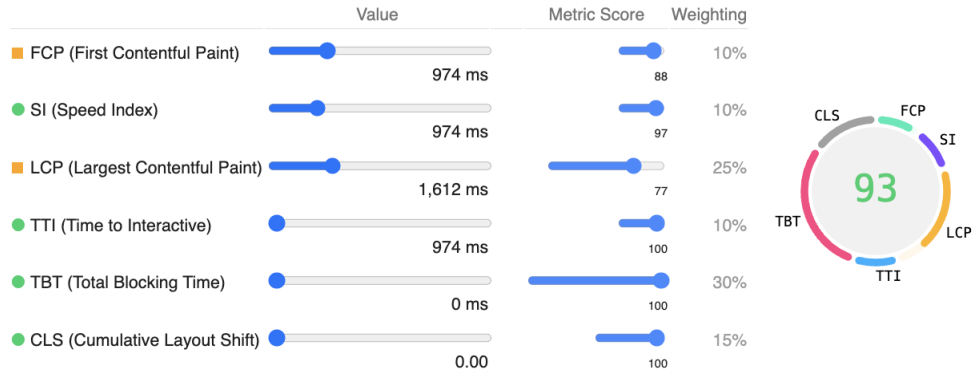


Figure 8.1: Distribution of Performance Metrics Scores for the Relation Visualiser Tool

Even though all the aforementioned metrics are essential to assess the responsiveness of the website, we are more interested in Time to Interactive (TTI) which measures the user's wait time before they can interact with the page. The performance score obtained by running the test on the Relation Visualiser tool page is shown in the figure above with a breakdown of the metric scores and their weight contribution to the final result. The TTI value obtained, which is approximately 0.9 seconds, indicates a reasonable page interactivity. However, to obtain an estimate of the entire system performance, we ran the same test across all the pages of our website. To see the proofs of the obtained scores, please refer to appendix (A.3).

Page	Accessibility Score	TTI
Home	96	0.9
Tools	95	0.9
Relation Visualiser Tool	93	0.9
Warshall Simulator Tool	93	1.0
Relation Explorer Tool	95	1.0
Reset Password	95	0.9
Login	95	0.9
Signup	95	1.0
Average Score	94.626	0.93

Table 8.1: Lighthouse Performance Scores and TTI for the Website Pages

The performance results obtained from Google Lighthouse shows that the system has an average performance score of approximately 94% and Time to Interactive (TTI) that is less than one second. This implies that our system is reasonably responsive, which achieves our non-functional requirements for performance (N.4 & N.5).

8.3 Accessibility Evaluation

To help evaluate the accessibility of the website, we have also utilised Google Lighthouse tool. This tool can compare the content of the page, against the Web Content Accessibility Guidelines (WCAG)² and compute the accessibility score accordingly. The computed score is based on various accessibility measures such as contrast ratio and compatibility with assistive technologies like screen readers. The score represents a weighted average of all the accessibility audits that has been carried out during the test. When running the accessibility test on our home page, we obtained a score of 94%. The score suggests that the page has strong accessibility features which meets the criteria set by WCAG. The figures below show the accessibility score as well as the passed audits that were used to compute the score.

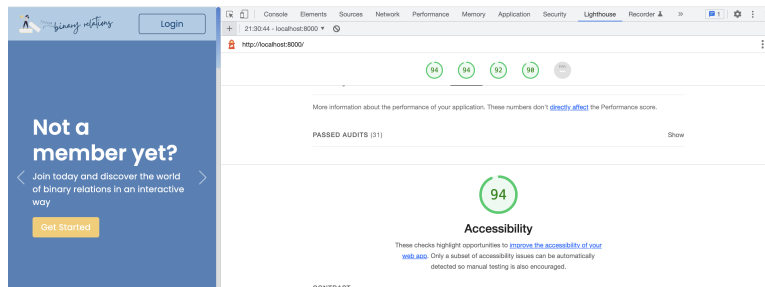


Figure 8.2: Lighthouse Accessibility Score for the Home Page

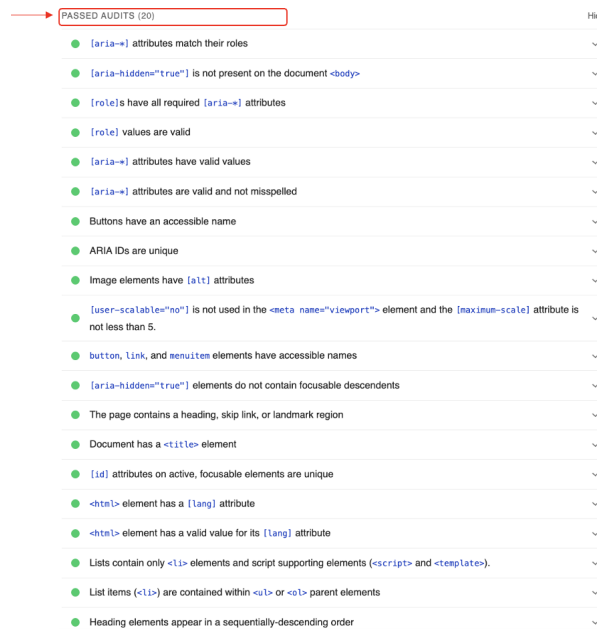


Figure 8.3: Lighthouse Passed Accessibility Audits for the Home Page

²<https://www.w3.org/WAI/WCAG20/versions/guidelines/wcag20-guidelines-20081211-a4.pdf>

However, to provide a comprehensive overview of our website’s accessibility, we ran the accessibility test on all our pages. We then computed the average score to estimate the accessibility score for the entire system. To see the proofs of the obtained scores, please refer to appendix (A.4).

Page	Accessibility Score
Home	94
Tools	95
Relation Visualiser Tool	94
Warshall Simulator Tool	94
Relation Explorer Tool	94
Reset Password	96
Login	92
Signup	92
Average Score	93.875

Table 8.2: Lighthouse Accessibility Scores for the Website Pages

Based on our accessibility evaluation, we can conclude that our website adheres to the WCAG accessibility guidelines to a good extent, which implies that our non-functional requirement (N.6) related to accessibility has been met.

8.4 Compatibility Evaluation

Due to the inconsistencies in different web-browsers and operating systems, it was necessary to conduct compatibility evaluation to guarantee that our website operates as expected and looks consistent across a wide range of browsers.

During development, we observed that some of the pages did not render correctly in an older version of the Safari browser. The pages did not follow the CSS rules that were specified resulting in a cluttered appearance of the pages. Upon further investigation and with the help of some online tools³, we realised that some of the CSS properties we have used were only supported by particular browser versions, namely the most recent ones. Consequently, we revised the CSS specifications that were used and modified it to account for different types and versions of browsers.

³<https://caniuse.com>

To help verify the system’s compatibility, we utilised BrowserStack⁴, a tool that allows developers to test the system on different browsers and operating systems. We ran the system on combinations of different operating systems and browsers and found that our website performs as expected and the pages were displayed correctly with a well-organised structure.

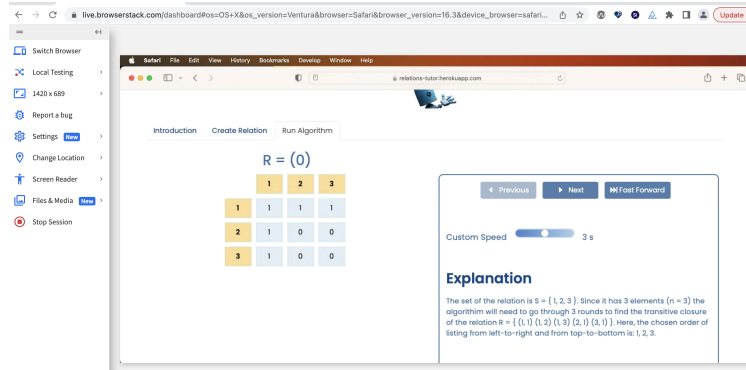


Figure 8.4: Testing the System on Safari Browser Using BrowserStack

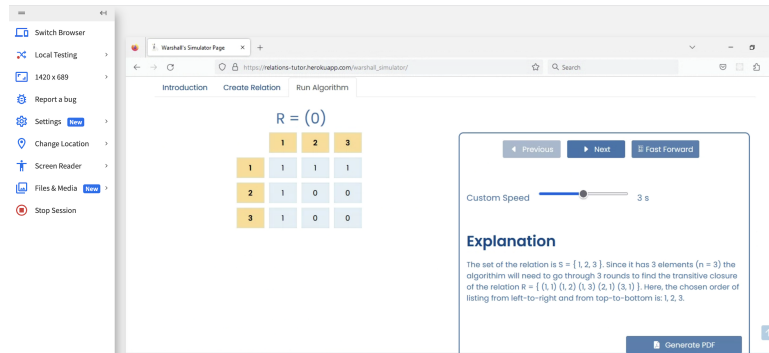


Figure 8.5: Testing the System on Firefox Browser Using BrowserStack

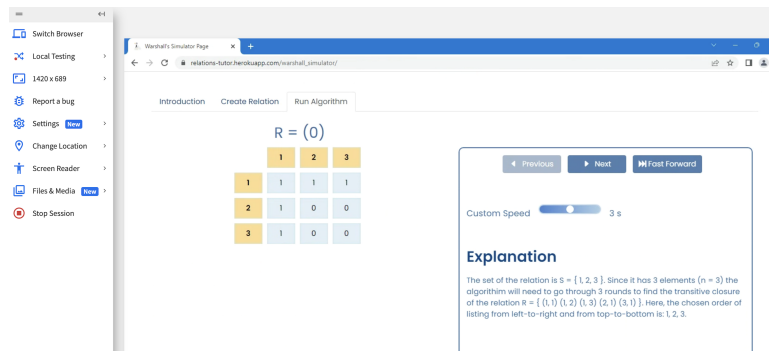


Figure 8.6: Testing the System on Chrome Browser Using BrowserStack

⁴<https://www.browserstack.com/compatibility-testing>

The table below summarises the results after running our compatibility tests.

Operating System	System Version	Browser Type	Browser Version	Compatible?
macOS	Ventura	Safari	16.3	Yes
macOS	Montrey	Safari	15.6	Yes
macOS	Big Sur	Safari	14.3	Yes
macOS	Catalina	Safari	13.3	Yes
macOS	Mojave	Safari	12.3	Yes
macOS	High Sierra	Safari	11.1	Yes
macOS	Sierra	Safari	10.3	Yes
Windows	Windows 11	Chrome	110	Yes
Windows	Windows 11	Opera	96	Yes
Windows	Windows 10	Edge	110	Yes
Windows	Windows 8	Firefox	110	Yes
Windows	Windows 7	Edge	109	Yes
Windows	Windows 7	Chrome	109	Yes

Table 8.3: Browser Compatibility

In conclusion, our compatibility evaluation demonstrates that our system is cross-browser and operating systems compatible. This implies that we have successfully met the non-functional requirement (N.3) for compatibility.

8.5 Critical Appraisal: Evaluating Strengths and Areas of Improvement

It is critical to perform an in-depth evaluation of the software system to identify its key strengths and potential areas of improvement. To thoroughly evaluate our system, we undertook various assessment methodologies, including manual and automated testing which evaluated the system against the functional requirements as well as performance, accessibility and compatibility assessments which served as an evaluation against our non-functional requirements. These evaluations provided us with valuable insights into the system's strengths and areas for improvement.

Among the most notable strengths of our system is its interactivity and responsiveness. The performance evaluation demonstrated that our system is highly performant, ensuring smooth and seamless user interactions. The system's ability to dynamically handle large relation sets without compromising responsiveness is a testament to its efficient performance. Furthermore, the website's accessibility evaluation has revealed that it is highly accessible, enabling the system to be used across a wide range of users including those with vision deficiencies.

The system's compatibility with different operating systems and browsers is another critical strength that can allow us reach a broader audience of learners. Finally, the system offers a vast range of features within each tool, coupled with a flexible learning environment that allow users to perform various operations on a relation with ease and efficiency.

However, it is important to acknowledge that no project is perfect. While the system demonstrated several strengths, there is always some room for improvement. One area that could be improved is related to handling sets larger than 50 elements. As we prioritise interactivity and responsiveness, we restricted the upper bound of the set size to 50 elements in order to optimise the system performance. Even though 50 elements could be sufficient for most regular learners to effectively experiment with binary relations in a variety of contexts, it can be inconvenient to more advanced learners who seek to experiment with data above this bound or compare the running time of the Warshall algorithm on varying set sizes. While optimising performance is essential, the website could explore ways to accommodate larger sets without compromising on speed and responsiveness. Furthermore, even though our website is fully functional on all devices including touch devices, the interactive directed graph in the Relation Visualiser tool is currently configured with only mouse events. Therefore, one way to improve the project would be to configure the graph with touch events in order to accommodate users using smartphones with touch screens.

In summary, the evaluation of the software has helped us to recognise its strengths such as interactivity and performance as well as opportunities for improvements including accommodating larger sets and configuring touch events. By acknowledging these strengths and limitations, we seek to further enhance the system and offer users a better learning experience.

Chapter 9

Professional and Ethical Issues

As part of the development process for this project, attribution of any third party content has been clearly stated in the report and source code. This includes proper citation of literature sources and explicit declaration of external libraries, open source code, icons and images that have been used in this project. In addition, the British Computing Society (BCS)¹ Code of Conduct's ethical standards has been carefully followed. This includes ensuring that the project is not built or used for harmful purposes and that all work is done within our professional competence. Moreover, the information presented on the website has been ensured to be accurate, credible and error-free to avoid misleading users. We also made considerable efforts to ensure that our learning platform follows effective approaches to teaching. This is achieved by prioritising the design of our learning tools to incorporate interactive and informative elements that can encourage engagement and simulate critical thinking. Finally, strong emphasis has been placed on accessibility. This involves ensuring that our website is user-friendly, easy to navigate, with a presentable, clear and readable content.

¹<https://www.bcs.org/media/2211/bcs-code-of-conduct.pdf>

Chapter 10

Conclusion and Future Work

10.1 Conclusion

The primary objective of this project was to develop an interactive web-based learning system to practice binary relations. With the aid of the literature, the three-way model for computer-initiated interaction has been utilised as the foundational principle to guide the creation of the system and optimise it for effective learning. The literature also highlights the significant impact of interactivity in the quality of student's learning, and this project acts as a demonstration of how it can be applied to the teaching of binary relations. With a focus on the creation of interactive learning tools, the system offers three distinct tools: a visualiser tool for visualising binary relations, a simulator tool for simulating the Warshall algorithm, and an explorer tool for exploring the various types and properties of binary relations. Through the adoption of the three-way model of computer-initiated interactions together with several interactive elements within our tools, the system was designed to make the student's learning more engaging and effective. Overall, the development of this project highlights the promising potential for interactive web-based learning systems in improving content delivery and promoting the students' active engagement in the field of Discrete Mathematics, and particularly in the area of binary relations.

10.2 Future Work

There are several valuable extensions that could be added for future versions of this project. Notably, the inclusion of game-based learning. A strong body of literature suggests the effectiveness of game-based learning in improving students' academic performance. Research shows that integrating gamification elements such as rewards, competitions and challenges in a learning setting can increase the students' engagement, motivation and performance [34][35]. Therefore, our system could be extended to include interactive games that are designed to help users test their understanding of binary relations in an exciting and stimulating way. For instance, we could incorporate the idea of a memory card game by presenting a set of cards and the user will need to match each relation with its alternative representation. Another way where we could incorporate game-based learning into the teaching of binary relations is to include a drag and drop matching game where users can drag and drop objects to a particular category. In the context of our project it could be that users have to drag and drop relations into an appropriate box representing a property of the relation, i.e. reflexive, symmetric etc. Some gamification elements such as difficulty levels and time constraints could be added to improve the learner's retention and engagement with the subject. However, to ensure that users get a new set of questions everytime they play a new game, a randomisation mechanism should be implemented. This mechanism ensures that questions are randomised with every new game, preventing the same set of questions from being repeated. Furthermore, to promote a collaborative learning environment, the project can be extended to include a discussion panel associated with each tool where users can actively post and discuss questions related to the material covered in the tool.

Moreover, as discussed in the implementation section, the project has extended the Hasse graph construction algorithm to minimise intersecting edges. However, it should be noted that the current implementation only eliminates instances of intersecting edges in most cases but not in all possible cases. The project could further improve the output of the graph by conducting a thorough research on algorithms that are optimised for this purpose. This is an interesting problem that links with several fields including computer science and engineering, where producing clear and easily readable data visualisations is essential for analysis and interpretation.

In conclusion, the extensions we have mentioned are a subset of many possible additions that could be integrated into our system to further enhance its interactivity and educational value.

References

- [1] Oxford University Press. Oxford learner's dictionary. <https://www.oxfordlearnersdictionaries.com/definition/english/interactivity>. Accessed on November 22, 2022.
- [2] Barry Dwyer. *Systems Analysis and Synthesis: Bridging Computer Science and Information Technology*. Morgan Kaufmann, 2016.
- [3] Peter C. Fishburn. *Mathematics of Decision Theory*. De Gruyter Mouton, 1972.
- [4] Gary Haggard, John S. Schlipf, and Sue Whitesides. *Discrete Mathematics for Computer Science*. Thomson Brooks/Cole, 2006.
- [5] David Liben-Nowell. *Connecting Discrete Mathematics and Computer Science*, chapter 8, page 385–438. Cambridge University Press, 2 edition, 2022.
- [6] K. Levasseur and A. Doerr. *Applied Discrete Structures*. Open Textbook Library. Lulu.com, 2012.
- [7] Anany V. Levitin. *Introduction to the Design and Analysis of Algorithms*. Addison-Wesley Longman Publishing Co., Inc., USA, 2002.
- [8] Rod Sims. Interactivity: A forgotten art? *Computers in Human Behavior*, 13(2):157–180, 1997.
- [9] Chris Evans and Khaled Sabry. Evaluation of the interactivity of web-based learning systems: principles and process. *Innovations in Education and Teaching International*, 40(1):89–99, 2003.
- [10] D. Laurillard. *Rethinking University Teaching: A Conversational Framework for the Effective Use of Learning Technologies*. RoutledgeFalmer, 2002.

- [11] Chris Evans and Khaled A. Sabry. Evaluation of the interactivity of web-based learning systems: principles and process. *Innovations in Education and Teaching International*, 40:89 – 99, 2003.
- [12] Young Ha and Hyunjoon Im. The role of an interactive visual learning tool and its personalizability in online learning: Flow experience. *Online Learning*, 24:205–226, 03 2020.
- [13] Chris Evans and Nicola J. Gibbons. The interactivity effect in multimedia learning. *Comput. Educ.*, 49(4):1147–1160, 2007.
- [14] Stanford University. Binary relation editor tool. <https://web.stanford.edu/class/archive/cs/cs103/cs103.1176/tools/relation-editor/>. Accessed on November 14, 2022.
- [15] University of Graz Cord Hockemeyer. Properties of relations tool. <https://r.qhelp.eu/Graz/Relations/>. Accessed on November 14, 2022.
- [16] Wheeling Jesuit University Marc Brodie. Exploring relations on sets tool. <https://demonstrations.wolfram.com/ExploringRelationsOnSets/>. Accessed on November 14, 2022.
- [17] Technische Universität München Aleksejs Voroncovs. The floyd-warshall algorithm tool. http://algorithms.discrete.ma.tum.de/graph-algorithms/spp-floyd-warshall/index_en.html. Accessed on November 14, 2022.
- [18] University of San Francisco David Galles. The floyd-warshall all-pairs shortest path tool. <https://www.cs.usfca.edu/~galles/visualization/Floyd.html>. Accessed on November 14, 2022.
- [19] Yashodha Sambrani, Apoorva M Mulmuttal, Pooja S Chandavar, Babita Naragund, and Aditi K Naik. She support(h)ers: A django based web application for women’s empowerment. *International Journal of Advanced Research in Computer and Communication Engineering*, 11:282–294, 2022.
- [20] Wilbert O. Galitz. *The Essential Guide to User Interface Design: An Introduction to GUI Design Principles and Techniques*. John Wiley Sons, Inc., USA, 2007.
- [21] Charles Miller. Aesthetics and e-assessment: The interplay of emotional design and learner performance. *Distance Education*, 32:307–337, 11 2011.

- [22] Kong-King Shieh and Chin-Chiuan Lin. Effects of screen type, ambient illumination, and color combination on vdt visual performance and subjective preference. *International Journal of Industrial Ergonomics*, 26(5):527–536, 2000.
- [23] Wouter Alberts and Thea Geest. Color matters: Color as trustworthiness cue in web sites. *Technical Communication*, 58:149–160, 05 2011.
- [24] Salimi Sohrab. Sprint - agile dictionary. <https://www.agile-academy.com/en/agile-dictionary/sprint/>. Accessed on April 1, 2023.
- [25] Project Management Institute (PMI). Technical debt. <https://www.pmi.org/disciplined-agile/agile/technicaldebt>, 2022. Accessed on April 1, 2023.
- [26] Mike Bostock and D3.js contributors. D3.js. <https://github.com/d3/d3>, 2011–present. Accessed on March 21, 2023.
- [27] The Cytoscape Consortium. Cytoscape.js. <https://github.com/cytoscape/cytoscape.js>, 2017–present. Accessed on March 21, 2023.
- [28] The jQuery Foundation. jquery, 2006–present. Accessed on March 21, 2023.
- [29] James Hall and yWorks GmbH. jspdf. <https://github.com/MrRio/jsPDF>, 2010–present. Accessed on March 21, 2023.
- [30] Inc. Twitter and Bootstrap contributors. Bootstrap. <https://github.com/twbs/bootstrap>, 2011–present. Accessed on March 21, 2023.
- [31] Craig Michael Thompson and qTip2 contributors. qtip2. <https://qtip2.com>, 2013–present. Accessed on March 21, 2023.
- [32] Django Central. Django + ajax : How to use ajax in django templates. <https://djangocentral.com/django-ajax-with-jquery/>. Accessed on April 1, 2023.
- [33] Can. Codepen. <https://codepen.io/turtlebuttocks/pen/RawLX0>, 2016.
- [34] Ünal Çakıroğlu, Betül Başbüyük, Mustafa Güler, Melek Atabay, and Bahar Yılmaz Memiş. Gamifying an ict course: Influences on engagement and academic performance. *Computers in Human Behavior*, 69:98–107, 2017.
- [35] Rubén Camacho-Sánchez, Aaron Rillo-Albert, and Pere Lavega. Gamified digital game-based learning as a pedagogical strategy: Student academic performance and motivation. *Applied Sciences*, 12:1–16, 11 2022.